

The Evolution of IBM's Quantum Information Software Kit (*Qiskit*): A Review of its Applications

Param Pathak¹, K Tarakeshwar², Syed Sufiyan Ali², Shalini Devendrababu³, and Adarsh Ganesan^{*4,5}

¹*QuantumAI Lab, Fractal Analytics, Mumbai, India 400063*

²Department of Computer Science, Birla Institute of Technology and Science (BITS) Pilani - Dubai Campus, Dubai, UAE 345055

³*QuantumAI Lab, Fractal Analytics, Gurugram, India 122003*

⁴Department of Electrical and Electronics Engineering, Birla Institute of Technology and Science (BITS) Pilani - Dubai Campus, Dubai, UAE 345055

⁵Department of Mechanical Engineering, Birla Institute of Technology and Science (BITS) Pilani - Pilani Campus, Pilani, India 333031

August 19, 2025

Abstract

Quantum computing is being increasingly adopted for solving classically intractable problems across various domains. However, the availability of accessible and scalable software frameworks remains essential for practical experimentation and adoption. IBM's open-source quantum computing toolkit *Qiskit*, has become a key player in this space by offering tools for circuit design, simulation, hardware execution, and domain-specific applications. This survey provides a systematic review of how *Qiskit* has evolved and what it has contributed to several critical fields including cryptography and cybersecurity, image and signal processing, climate science and energy applications, and finance. We show how *Qiskit* facilitates hybrid classical-quantum workflows and enables the deployment of algorithms on physical quantum hardware through error mitigation and modular integration approaches. Our exploration covers several key applications, including quantum key distribution, climate simulation, and quantum-enhanced portfolio optimization, while providing practical insights into their implementation. This work also covers the framework's technical structure and current limitations associated with scalability and reproducibility. By bringing together developments that have been scattered across different areas, this work serves as a reference point for researchers and practitioners who want to understand or contribute to *Qiskit*-enabled quantum computing.

1 Introduction

Quantum computing (QC) [1] offers transformative potential for various fields including cryptography and material science by addressing problems that classical computers cannot feasibly solve [2]. For instance, quantum algorithms like Shor's factorization method [3] and Grover's search algorithm [4] can provide quicker solutions for important computational problems. However, getting practical quantum advantage isn't just about building better hardware. We also need software frameworks that are easy to use and can connect theory with practice.

Qiskit [5], an open-source QC library developed by IBM, has become indispensable in this ecosystem. *Qiskit* handles all the tricky hardware parts and provides modular tools for circuit design, simulation, and execution,

*Corresponding author: adarsh@dubai.bits-pilani.ac.in

helping researchers, developers, and industries get to quantum resources without any hassle [6]. Specialized libraries like *Qiskit* Nature [7] and *Qiskit* Machine learning [8] create a versatile platform that drives innovation in various fields. *Qiskit* was launched in 2017 as a basic quantum circuit construction toolkit, but has now grown into a comprehensive ecosystem that supports research [9], education [10], and industrial applications [11]. While its initial versions focused on core components like Terra for circuit design and Aer for simulation, later developments introduced specialized modules such as Nature for quantum chemistry and Finance for financial applications [12]. By 2024, *Qiskit* attracted more than 600,000 users around the world, drawing academic institutions, startups, and Fortune 500 companies for drug discovery [13], financial modeling [14], and cybersecurity applications [15]. *Qiskit*'s integration with IBM quantum hardware ranging from 5-qubit prototypes to sophisticated 433-qubit Osprey processors [16] demonstrates its effectiveness as a connecting framework between algorithmic theory and current near-term QC systems [17].

Although *Qiskit* has seen widespread adoption, a systematic review of its applications is missing. This paper takes a critical look at how *Qiskit* helps move practical QC solutions forward across four different interdisciplinary areas: cryptography and cybersecurity; image and signal processing; climate science and energy applications; finance and risk analysis. By bringing together research efforts that have been scattered across different areas, this work aims to create a unified framework for understanding how *Qiskit*'s modular architecture covering circuit design, simulation, and domain-specific libraries allows researchers to turn theoretical quantum algorithms into actual experimental workflows.

This work can be outlined as follows:

- Explains how *Qiskit* addresses challenges in the aforementioned areas by focusing on the implementations of hybrid quantum-classical algorithms such as the Variational Quantum Eigensolver (VQE) [18] and the Quantum Approximate Optimization Algorithm (QAOA) [19].
- Explores the technical workflows that allow *Qiskit* to connect theoretical concepts with actual implementation, particularly through its integration with IBM quantum hardware for noisy intermediate-scale quantum (NISQ) experiments.
- Discusses the constraints present in current *Qiskit*-driven approaches, especially when dealing with scalability issues, keeping qubit coherence intact, and ensuring reproducibility of results in real-world settings.

The paper is organized as follows. Section 2 looks at the architecture of *Qiskit*; Section 3 discusses the major applications of *Qiskit*. In subsection 3.1, we discuss the cryptography and cybersecurity applications viz. quantum key distribution (QKD) and quantum random number generation (QRNG). In subsection 3.2, we showcase the use of *Qiskit* in image and signal processing relating to medical imaging and telecommunications. In subsection 3.3, we cover climate and energy applications including climate modeling and energy optimization. In subsection 3.4, finance applications are discussed in the context of portfolio optimization (PO) and risk assessment. Section 4 summarizes the main findings.

2 *Qiskit* Architecture and Core Components

While QC has incredible theoretical potential, it really needs solid software frameworks that can turn abstract algorithms into something executable. *Qiskit* tackles this challenge with a modular architecture that hides the complexity of hardware while still giving researchers and developers the flexibility they need. *Qiskit* is built as a layered system that has four main components: *Terra*, *Aer*, *Ignis*, and domain-specific libraries.

Terra (which means "Earth" in Latin) forms the foundation of *Qiskit*. It lets users design quantum circuits with familiar components like gates and qubits, and then optimize these circuits for specific hardware through

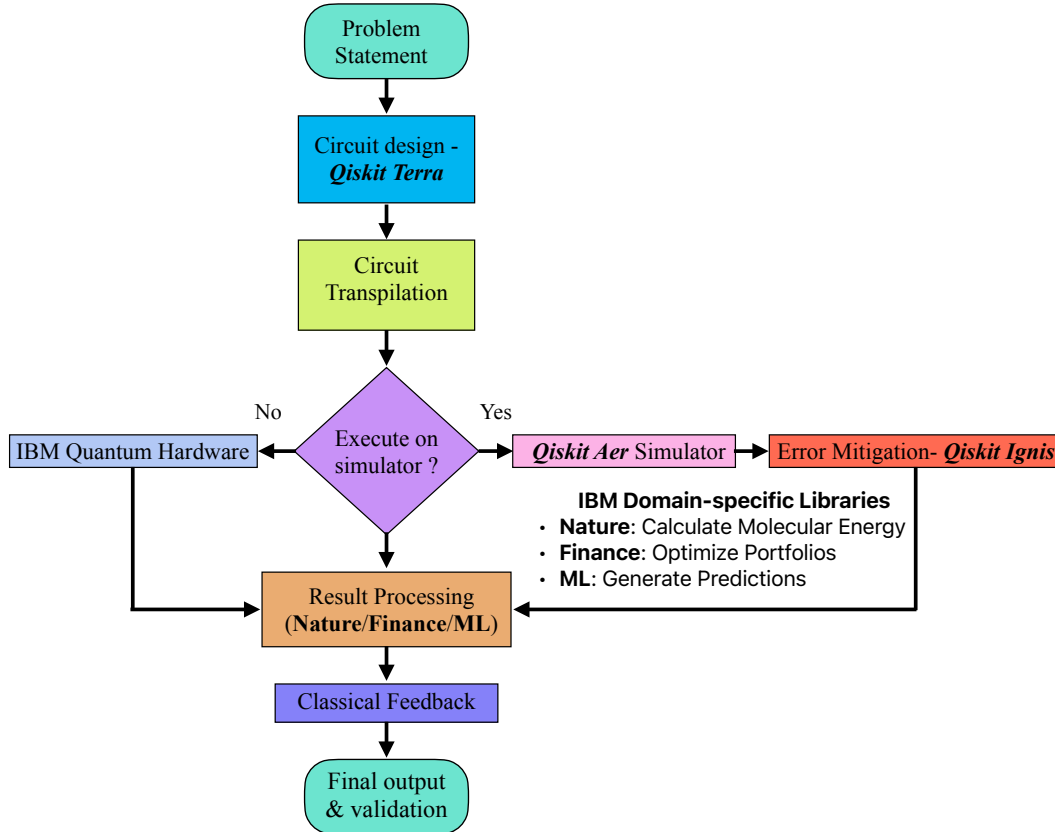


Figure 1: The workflow begins with defining a domain-specific problem. A quantum circuit is designed and optimized using *Qiskit Terra*. The circuit is executed either on a *Qiskit Aer* simulator or a real hardware (IBM Quantum). Results are post-processed with domain-specific libraries and refined via classical feedback. Final outputs are validated against classical benchmarks.

transpilation. This process is similar to translating a high-level blueprint into machine-readable instructions, ensuring proper functioning across simulators and real quantum devices.

Terra works together with *Aer* (Latin for "Air"), a high-performance simulator framework that mimics quantum hardware. *Aer* offers multiple simulation approaches, including statevector simulations for ideal QCs and noisy simulations that recreate the decoherence and gate errors seen in real devices.

Ignis (Latin for "Fire") helps deal with NISQ device limitations by providing tools for error characterization, mitigation, and calibration. *Ignis* includes modules that measure and reduce errors, for instance, fixing misread qubits or extending their short-lived quantum states to make the results of today's imperfect devices more reliable.

Beyond these foundational layers, *Qiskit* offers domain-specific libraries including Nature, Machine Learning (ML), Optimization, and Finance that make specialized tasks much simpler. Researchers can use *Qiskit Nature* for running molecular simulations, while *Qiskit Finance* allows stock portfolio modeling. *Qiskit ML* supports hybrid quantum-classical models using tools like quantum kernels and neural networks (NN). Through IBM Quantum Provider, users get cloud access to hardware like the 127-qubit IBM Quantum Eagle. The comprehensive documentation and a presence of an active community make QC accessible to people across various skill levels.

Fig. 1 shows the workflow of *Qiskit* for QC applications. The process begins with a problem statement, where circuits are designed using *Qiskit Terra* and then transpiled to ensure hardware compatibility. Users can choose

between IBM Quantum Hardware and *Qiskit Aer* Simulator (which includes error mitigation through *Ignis*). The decision to use *Qiskit Aer* simulators instead of IBM Quantum hardware is typically based on research objectives and practical constraints. Simulators provide controlled test environments where researchers can work with perfect fidelity and customize noise models as needed, making them particularly effective for rapid algorithm development and validation of theoretical concepts. Hardware execution becomes essential when researchers need to understand how algorithms actually behave under real quantum noise, coherence limitations, and connectivity constraints, particularly for applications seeking practical quantum advantage or when evaluating error mitigation strategies. This strategic approach allows users to utilize resources more efficiently while keeping a good balance between theoretical exploration and experimental validation. The results from the simulator/hardware are processed using domain-specific libraries (Nature, Finance, or ML) and are iteratively refined. The final outputs are validated against classical benchmarks.

3 Major Application Domains

The modular architecture of *Qiskit* provide a platform for researchers and developers to execute quantum algorithms for solving real-world problems. By combining domain-specific libraries with quantum workflows, it enables experimental execution of theoretical ideas. This section highlights how *Qiskit* drives advancements across diverse fields.

3.1 Cryptography and Cybersecurity

While today’s cryptographic systems keep our communications secure and data protected, quantum algorithms are becoming a threat to current encryption methods [20, 21]. *Qiskit* provide tools for building quantum-resistant protocols and running simulations to test the vulnerability of our current systems to attacks. This section looks at how *Qiskit* enables connection between quantum theory and real-world security applications, offering protection against current and future anticipated threats to our digital infrastructure.

3.1.1 Quantum Key Distribution

QKD [22] uses the no-cloning theorem [23] and observer-induced state collapse [24], to allow exchange of secure keys between parties. Any eavesdropping attempt can manifest as a quantum measurement perturbing quantum states and hence can be detected. The Bennett-Brassard 1984 (BB84) scheme [25] is notable among developed protocols due to its potential for practical implementation and its foundational importance in the field.

The BB84 protocol is demonstrated in the following code (lines 1–36). Alice encodes her classical bits into quantum states using two mutually unbiased bases. For each qubit (lines 8–13), Alice applies a Quantum-NOT gate (X -gate) (line 11) to encode a classical 1 as the state $|1\rangle$ in the Z -basis or uses a Hadamard-gate (H -gate) (line 13) to prepare superpositions ($|+\rangle = H|0\rangle$, $|-\rangle = H|1\rangle$) in the X -basis. This choice of applying either an X -gate or a H -gate depends on her randomly chosen basis (`alice_bases`). Bob then independently chooses his own measurement bases (`bob_bases` line 16), and applies H -gates (line 21) to undo the encoding when his basis matches Alice’s. The measurement step (M -gate) (line 22) causes qubits to collapse into a classical bit, governed by quantum state collapse mechanism. The protocol then sifts the key (lines 30–36) by comparing Alice’s and Bob’s bases (`alice_bases[i] == bob_bases[i]`, line 34), and keeps only those bits where the bases align. When Alice encodes a bit in the X -basis using an H -gate and Bob measures by applying his own H -gate in the same basis, the measurement reverses the superposition, giving us deterministic outcomes: $|+\rangle \rightarrow 0$ or $|-\rangle \rightarrow 1$ to enable secure key agreement. During eavesdropping attempts, the quantum states are perturbed. This results in an increase in the quantum bit error rate (QBER), which can be detected during classical post-processing.

```
1 # Alice's random bits and bases (4 qubits)
2 alice_bits = np.random.randint(2, size=4)
3 alice_bases = np.random.randint(2, size=4)
```

```

4
5 # Create quantum circuit
6 qc = QuantumCircuit(4, 4)
7
8 # Alice prepares qubits
9 for i in range(4):
10     if alice_bits[i] == 1:
11         qc.x(i)
12     if alice_bases[i] == 1:
13         qc.h(i)
14
15 # Bob's random bases
16 bob_bases = np.random.randint(2, size=4)
17
18 # Bob measures
19 for i in range(4):
20     if bob_bases[i] == 1:
21         qc.h(i)
22 qc.measure(range(4), range(4))
23
24 # Simulate
25 backend = Aer.get_backend('qasm_simulator')
26 job = execute(qc, backend, shots=1)
27 result = job.result()
28 bob_results = list(result.get_counts().keys())[0][::-1]
29
30 # Key sifting
31 alice_key = []
32 bob_key = []
33 for i in range(4):
34     if alice_bases[i] == bob_bases[i]:
35         alice_key.append(alice_bits[i])
36         bob_key.append(int(bob_results[i]))

```

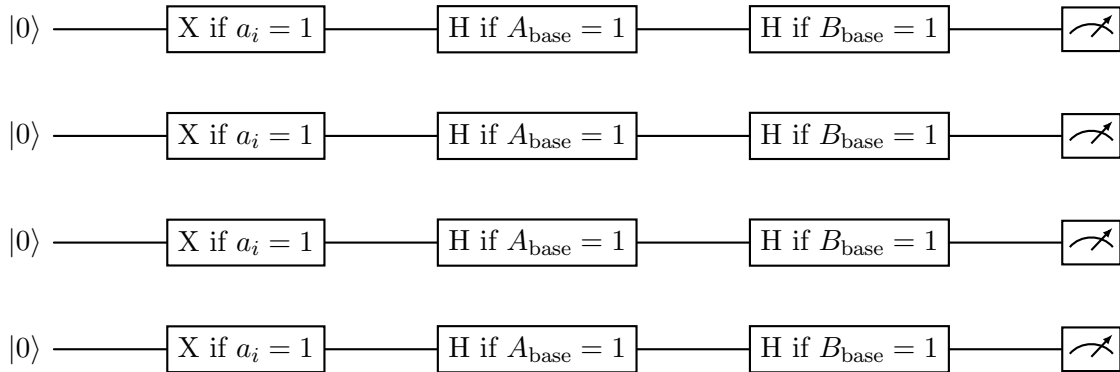


Figure 2: BB84 QKD Protocol. Alice encodes her classical bits a_i and applies her basis choice. Bob applies his basis choice before measurement. The shared key is constructed from positions where both used the same basis.

The generated circuit, as shown in Fig. 2, illustrates the BB84 protocol's quantum steps for individual qubits. Alice encodes her classical bit 1 using an X -gate, then applies an H -gate when choosing the diagonal basis. Bob applies H -gate when measuring in the diagonal basis, followed by M -gate. When both Alice and Bob use the same measurement bases, either Alice and Bob both apply or do not apply H -gates. This results in predictable and reliable outcomes. However, when their bases are different, 'only' one of them applies H -gate, creating an imbalance. This brings in randomness through quantum superposition. This intentional sensitivity to mismatches is

particularly important for secure key generation. During eavesdropping attempts, the quantum states are perturbed, resulting in elevated error rates in basis reconciliation. This allows Alice and Bob to identify potential interference.

Recent studies show how *Qiskit* functions as a practical platform for designing and validating quantum cryptographic protocols. We examine several important studies that employ *Qiskit* for various applications in quantum cryptography and cybersecurity.

Mohanty *et al.* [26] developed an identity-based signature scheme for secure email communication, where quantum states are prepared and verified in a way that makes any eavesdropping detectable. Their protocol characterizes the interference of quantum states through the equation:

$$\langle \psi | U(\theta) | \psi \rangle = \cos \theta, \quad (1)$$

where $|\psi\rangle$ denotes the prepared quantum state, θ represents the rotation angle applied to the state, and $U(\theta)$ is a unitary rotation operator parameterized by θ . The inner product $\langle \psi | U(\theta) | \psi \rangle$ quantifies the overlap between the original and rotated states, ensuring that deviations introduced by an adversary are measurable.

Similarly, Faleiro and Goulão [27] developed a device-independent quantum authorization protocol based on the Clauser–Horne–Shimony–Holt (CHSH) game. Their approach exploits non-local correlations quantified by the CHSH parameter:

$$S = E(a, b) - E(a, b') + E(a', b) + E(a', b'), \quad (2)$$

where $E(x, y)$ denotes the expectation value of the product of measurement outcomes x and y . a and a' represent two possible measurement settings for Alice, while b and b' represent two possible measurement settings for Bob. Classical systems satisfy $S \leq 2$ while ideal quantum devices can reach up to $2\sqrt{2}$. This equation confirms the security proof by verifying the presence of genuine quantum correlations.

Pereira da Silva [28] showcased practical circuit implementations for cryptographic primitives. The central operation in these circuits is the creation of superposition states using the H -gate, which is crucial for protocols such as BB84 and other QKD schemes.

Das *et al.* [29] harnessed *Qiskit* to build quantum equivalents of classical hashing algorithms (SHA-1, MD5, SHA-256) for secure communication protocols. They employed *Qiskit*'s circuit libraries to create quantum versions of essential operations, like modular addition, which enables chaining variable updates in hash functions. A key element in their implementation was the quantum modular adder, which operates according to:

$$|A\rangle|B\rangle|0\rangle \rightarrow |A\rangle|B\rangle|(A + B) \mod 2^n\rangle, \quad (3)$$

where $|A\rangle$ and $|B\rangle$ denote quantum registers encoding the n -bit integers A and B , respectively, the third register $|0\rangle$ is an ancillary register initialized to the all-zero state, n determines the bit-length of the hash (e.g., $n = 32$ for MD5), and the operation $(A + B) \mod 2^n$ denotes addition modulo 2^n . This circuit enabled efficient computation of iterative steps like:

$$cv_i = (cv_{i-1} + \text{block}_i) \bmod 2^n, \quad (4)$$

where cv_i denotes the chaining value at iteration i , cv_{i-1} is the chaining value from the previous iteration, and block_i represents the i -th message block in the hashing process. This is a core process in hash generation.

IBM's research into its *Qiskit* toolchain [30] provides additional evidence of the framework's effectiveness in cybersecurity applications. Their work assesses circuit performance by calculating the fidelity of quantum states, given by:

$$F = \langle \psi | \rho | \psi \rangle, \quad (5)$$

where F quantifies how closely the experimentally produced state ρ approximates the ideal state $|\psi\rangle$.

Perepechaenko and Kuang [31] showed the working principle of symmetric encryption scheme using the Quantum Permutation Pad (QPP) on IBM quantum systems through *Qiskit*. They used 2-qubit permutation gates from the symmetric group S_4 , where each permutation gives $\log_2(24) \approx 4.58$ bits of entropy. They used a pad of 28 permutations to ensure 128-bit security. Plaintext blocks (such as 2-bit image data) were encrypted using permutation matrices such as:

$$M = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad (6)$$

with decryption employing Hermitian conjugates. Despite classical transmission of measured ciphertext, this work highlights *Qiskit*'s role in practical quantum-secure communication.

3.1.2 Quantum Random Number Generation

QRNG [32] takes advantage of the inherently probabilistic nature of quantum mechanics to create genuine randomness, which is different from classical pseudo-random number generators that depend on deterministic algorithms. High-quality random numbers serve as the foundation for cryptographic applications, such as key generation, nonce creation, and secure initialization vectors. *Qiskit* offers an accessible framework for implementing QRNG protocols that use quantum superposition and measurement to generate statistically unpredictable sequences. An implementation of QRNG using *Qiskit* shows how quantum superposition can be used to generate true random numbers for cryptographic applications.

The implementation we show below (lines 1–26) demonstrates QRNG that leverages *Qiskit*'s quantum circuit capabilities. We begin the code by establishing parameters for the random number generation. `n_bits` sets the bit-length of each random number and `n_numbers` tells us how many random numbers we need to generate (lines 2–3). Our `generate_random_number` function constructs a quantum circuit with `n_bits` qubits (line 7), where we initialize each qubit in the $|0\rangle$ state. Following this initialization, we apply H -gates to all qubits (line 11), which creates a uniform superposition. We use this quantum superposition as a source of true randomness, because quantum mechanics tells us that measurement outcomes are fundamentally unpredictable. After we take the measurement (line 12), the function turns the binary result into an integer (lines 15–20) such that we end up with a random number somewhere between 0 and $2^{n_bits} - 1$. The main routine (lines 22–26) runs this quantum circuit multiple times to generate several random numbers.

```

1  # QRNG parameters
2  n_bits = 8      # Number of bits for each random number
3  n_numbers = 10  # Number of random numbers to generate
4
5  # Function to generate a single random number using quantum circuit
6  def generate_random_number(n_bits):
7      qc = QuantumCircuit(n_bits, n_bits)
8
9      # Create superposition of all possible states
10     # Each qubit has 50% chance of being 0 or 1
11     qc.h(range(n_bits))
12     qc.measure(range(n_bits), range(n_bits))
13
14     # Execute circuit on simulator
15     job = execute(qc, backend, shots=1)
16     result = job.result()
17     counts = result.get_counts(qc)
18
19     # Convert binary measurement outcome to integer
20     return int(list(counts.keys())[0], 2)

```

```

21
22 # Generate multiple random numbers
23 random_numbers = []
24 for _ in range(n_numbers):
25     random_number = generate_random_number(n_bits)
26     random_numbers.append(random_number)

```

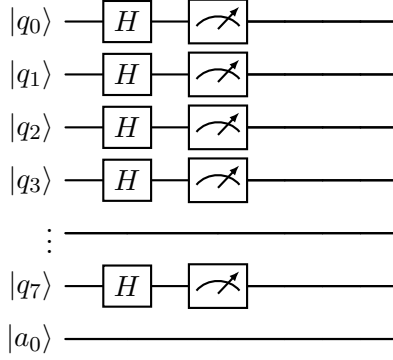


Figure 3: QRNG Circuit. Eight qubits are initialized in $|0\rangle$, put into superposition using H -gates, then measured to generate an 8-bit random number. Each measurement yields a truly random bit with 50% probability of being 0 or 1.

Fig. 3 shows the quantum circuit to generate an 8-bit random number. The circuit configuration has eight qubits, with each initialized in the $|0\rangle$ state. Following initialization, H -gates are applied across all qubits, resulting in a uniform superposition state where each qubit exhibits equal probability amplitudes for the $|0\rangle$ and $|1\rangle$ basis states upon measurement. Following this, M -gates are applied on all qubits, causing the superpositions to collapse into definite classical bit values. As a result, we get an 8-bit string which represents a truly random number between 0 and 255, where each possible value has equal probability. While the circuit may look simple, it possesses genuine cryptographic strength. Classical pseudo-random number generators rely on deterministic algorithms, but the randomness generated here arises from the fundamental uncertainty that exists in quantum measurement. This offers an exceptional entropy source for cryptographic applications. Below are some of the studies that have used *Qiskit* for QRNG.

In the work [33], *Qiskit* is used to simulate a quantum-enhanced Monte Carlo method employing a Permuted Congruential Generator (PCG) for pseudo-random number generation (PRNG). The authors implement PCG on a quantum circuit using modular arithmetic operations; for instance, the recurrence relation,

$$\tilde{x}_{n+1} = (a \tilde{x}_n + c) \bmod m, \quad (7)$$

where a is the multiplier, c is the increment, m is the modulus of the linear congruential generator in the PCG, and $\tilde{x}_n$ is the n -th state of the background sequence in the PCG before permutation. The parameters satisfy $a > 0$, $c \geq 0$, $m > 0$, and $0 \leq \tilde{x}_0 < m$. The output pseudo-random number is obtained by applying a permutation function $g(\tilde{x}_n)$ to $\tilde{x}_n$. This process is combined with bitwise permutations to reduce qubit overhead. They demonstrate this by integrating a trigonometric function:

$$I = \frac{1}{\theta^{N_{\text{var}}}} \int_0^\theta \sin^2 \left(\sum_{i=1}^{N_{\text{var}}} x_i \right) dx_1 \cdots dx_{N_{\text{var}}}, \quad (8)$$

where θ is the integration range, N_{var} is the number of variables, and x_i represents the i -th integration variable. Pseudo-random number sequences are encoded into quantum states via controlled rotations, and *Qiskit*'s simulator validates the approach with amplitude estimation, achieving an error scaling as $\mathcal{O}(N_{\text{orac}}^{-1})$, where N_{orac} is the number

of oracle calls. This shows how *Qiskit* functions as an effective prototyping platform for quantum algorithms using PRNG within high-dimensional financial applications.

Sinai *et al.* [34] employed *Qiskit* to build a quantum-secure transaction shuffling protocol. They constructed a QRNG using IBM Quantum Composer, leveraging superposition through H -gates to produce random indices (such as 5 qubits $\rightarrow$ 0–31 range) for reordering blockchain transactions. Through integration with decentralized nodes, *Qiskit*’s software development kit enabled validation of the protocol’s performance reached 25ms latency for 8192 transactions. This illustrates *Qiskit*’s capacity for enabling quantum-resistant, decentralized solutions against MEV attacks.

Germain *et al.* [35] showed how *Qiskit* reset functionality could address earlier QRNG limitations, where generating each bit required its own dedicated qubit. Their Python *grandom* function created multi-phase quantum circuits that could reuse qubits through reset operations, allowing them to produce more bits than the number of available qubits (for example, getting 9-bit outputs from just 3 qubits). They placed qubits into superposition to ensure equal probability during measurement. The random numbers they generated passed the National Institute of Standards and Technology (NIST) randomness tests with ”very strong” results, that outperformed classical methods like Python’s *random* and matching NIST’s pi-based benchmarks.

Kumar *et al.* [36] developed a 24-qubit QRNG using *Qiskit*, making use of its *QuantumCircuit* module to apply H -gates iteratively (1–9 layers per qubit) and create superposition states. They also used *Qiskit*’s *transpile()* function to optimize gate sequences and reduce decoherence, while the *ibmq-qasm_simulator* handled 65,536 shots to ensure statistical rigor. This approach produced outputs with 0.00075 min-entropy and 0.99952 worst-case entropy. *Qiskit*’s backend integration enabled real-time circuit validation, and henceforth achieving 2.4M bits/sec throughput along with 99.92% gate fidelity. The results passed all NIST SP 800-22 tests, including a p -value of 0.978 for Binary Matrix Rank, which effectively shows *Qiskit*’s capabilities for developing scalable, high-speed quantum randomness generation.

Li *et al.* [37] developed a source-independent QRNG using IBM’s cloud-based superconducting QCs. To address the inherent hardware noise issues, they adapted a protocol that was originally designed for optical source-independent QRNG implementations. Using *Qiskit*, they built circuits involving gates with rotation around y -axis ($RY(\pi/2)$) to prepare superposition states and then employed *Toeplitz-matrix hashing* for post-processing. Key to their approach was estimating the superposition state preparation error e_z under asymmetric readout errors ($r_0 \neq r_1$), modeled via:

$$K_{\text{final}} = (1 - r_1 + r_0) [n_z - n_z H(e_z)] - t_e, \quad (9)$$

where $H(e_z)$ is the Shannon entropy of the preparation error, n_z is the number of valid measurement outcomes in the Z -basis, t_e is the number of bits consumed for error correction and privacy amplification during post-processing, and K_{final} represents the final number of extractable certified random bits. *Qiskit*’s *transpile()* function worked to optimize gate sequences for mitigating decoherence, while backend tools ran 8.19M shots across 24-qubit devices (*IBMQ_5_yorktown*, *IBMQ_lima*). This generated 741,006 certified random bits with NIST-compliant p -values > 0.01 and near-zero autocorrelation ($|\rho| < 0.0038$). The real-time readout error calibration and circuit validation capabilities in *Qiskit* turned out to be critical for reaching 99.92% gate fidelity and making scalable randomness extraction possible.

Orts *et al.* [38] designed a *Qiskit*-based quantum circuit to generate interval-specific random numbers $[0, B)$ on NISQ devices, combining Grover’s algorithm with a T-optimized comparator. The comparator uses temporary logical-AND gates to achieve $4N$ T-count and $2N$ T-depth for N -digit numbers, requiring only $2N + 1$ qubits. Using *Qiskit*’s transpilation approach, the circuit applies $I = \left\lfloor \frac{\pi}{4} \sqrt{\frac{2^N}{B}} \right\rfloor$ iterations, which results in $> 80\%$ uniformity for intervals that cover $\geq 15.6\%$ of 2^N on IBM’s *ibmq-qasm_simulator*, although noise does reduce TestU01 scores to around ~ 0.57 . The design managed to achieve 2.4M bits/sec throughput, showing how well *Qiskit* can balance gate optimization and qubit connectivity for scalable QRNGs.

Jacak *et al.* [39] developed QRNG that uses multi-qubit entanglement for public verifiability, working with the

state $|\Psi\rangle$, where testing one sequence’s randomness through NIST SP-800-22 validates all the others. Their JUR02 prototype worked with photodiode shot noise and achieved 1 Mb/s throughput with roughly $\sim 0\%$ bias, successfully passing NIST/Dieharder tests without requiring any post-processing, while JUR01 required von Neumann whitening to correct bias problems. The experiments conducted on 100 MB samples showed that both ComScire QRNG and Mathematica PRNG successfully passed all NIST tests, which demonstrates how quantum protocols can effectively resist classical predictability. The entanglement-based framework enabled verification of public randomness using $k - 1$ secure sequences, showing its potential for cryptographic applications.

3.2 Image and Signal Processing

Classical ways of image and signal processing require high resources to do matrix operations such as Fourier transforms, filtering and convolution, which becomes inefficient as the volume of data and complexity grows. When dealing with real-time applications which have high resolution scenarios, the traditional approaches often face limitations when dealing with metrics like speed, memory and scalability [40]. QC can help overcome these problems by basic principles such as superposition and entanglement which processes all the information in parallel, and hence significantly reduces the time taken to complete these complex tasks [41]. *Qiskit* can help overcome the challenges faced here by encoding the classical image data into quantum states and thereupon implementing quantum circuits to process the data [42]. This section will focus on how *Qiskit* driven image and signal processing techniques are being applied across different sectors and industries like medical imaging and telecommunications while also showing the impact as well as its future potential.

3.2.1 Medical Imaging

Common medical imaging methods like magnetic resonance imaging (MRI) and computed tomography (CT) involve time-consuming image construction algorithms [43]. Through quantum medical imaging, one can harness features such as quantum parallelism and entanglement to process information at an exponentially faster rate [44]. Using linear algebra and optimization-specific quantum algorithms, it is possible to accelerate the reconstruction phase of the imaging process, aiding in faster understanding of the smaller anatomical variations [45, 46].

Flexible Representation of Quantum Images (FRQI) is one of the most used methods for representing images in the quantum form. Using FRQI, we can convert classical images of medical diagnostics into quantum states for more efficient processing [42]. In the FRQI scheme, the pixel intensities are encoded as an angle of rotation θ on the color qubit through controlled- R_y (CR_y) gates. $\theta = 0$ represents a fully black pixel corresponding to $|0\rangle$ state, whereas $\theta = \frac{\pi}{2}$ indicates a fully white pixel corresponding to the superposition of $|0\rangle$ and $|1\rangle$ states. Angles between $\theta = 0$ and $\theta = \frac{\pi}{2}$ correspond to grayscale intensities, allowing for an efficient grayscale representation of image [47]. This encoding enables the application of powerful quantum algorithms, in the field of medical imaging, to perform noise removal, edge extraction and segmentation, in principle with an exponential improvement in speed. Using software like *Qiskit*, it is possible to simulate FRQI implementations and to run quantum gates to transform images or to more generally run hybrid quantum-classical pipelines on a quantum simulator or on real quantum systems [48].

The implementation of FRQI is shown below. We use $\theta = 0$ to show minimum intensity pixels (line 1). In lines 5-6, we establish superposition of pixel positions. Here, the H -gates are applied to qubits 0 and 1, allowing them to represent the four pixel positions $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$. After establishing this, we sequentially encode the intensity of these pixels. In lines 10-20, we use CR_y -gates and CX -gates for pixel 1 to entangle the position of pixel $|00\rangle$ with its corresponding intensity. In line 22, pixel 2 is encoded by applying $X(1)$ to flip qubit 1 into the state $|01\rangle$ to start with. In lines 24-30, we repeat the same entangling structure. The qubits are then measured (line 32) and the corresponding circuit is drawn (line 34).

```
1 theta = 0 # pixel intensity (black) -> R_y(0)
2
```

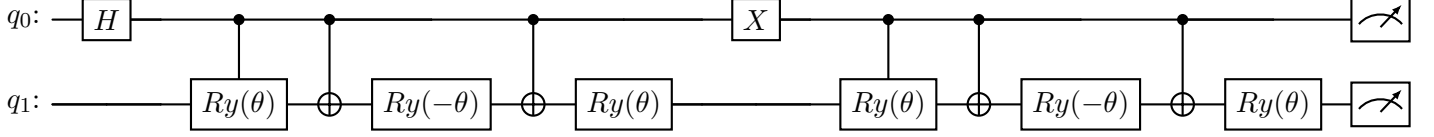


Figure 4: FRQI encoding circuit with two pixels. The circuit creates superposition over addresses, then encodes pixel intensities using CR_y Rotation with CX -gates for each pixel address.

```

3 qc = QuantumCircuit(2) # q0: address qubit, q1: intensity qubit
4
5 # Create equal superposition over 2 addresses
6 qc.h(0) # H|0> -> (|0> + |1>)/sqrt(2)
7
8 qc.barrier(label="Pixel_0")
9
10 # Encode pixel at address |0> using standard FRQI encoding block
11 # CRY(theta) - CX - CRY(-theta) - CX - CRY(theta) = net controlled-R_y(theta)
12
13 qc.cry(theta, 0, 1)
14 qc.cx(0, 1)
15 qc.cry(-theta, 1, 1)
16 qc.cx(0, 1)
17 qc.cry(theta, 1, 1)
18
19 qc.barrier(label="Pixel_1")
20
21 # Step 3: Flip address qubit to target |1>
22 qc.x(0) # Now q0 = |1>
23
24 # Step 4: Encode pixel at address |1> using same FRQI block
25 qc.cry(theta, 0, 1)
26 qc.cx(0, 1)
27 qc.cry(-theta, 1, 1)
28 qc.cx(0, 1)
29 qc.cry(theta, 1, 1)
30
31 # Step 5: Final measurement of both qubits
32 qc.measure_all()
33
34 qc.draw('mpl')

```

The circuit shown in Fig. 4 shows the application of FRQI technique which is used here to encode a two pixel grayscale image. One qubit (q_0) is used to represent the pixel positions, while the other qubit (q_1) is used to encode brightness. It starts with H -gate being applied into q_0 , resulting in superposition of both addresses of pixels. CR_y rotations are then applied on q_1 , which is used by the address qubit to map pixel intensities. Then a controlled 3-gate rotation is formed in the pattern $CR_y(\theta)$, CX , $CR_y(-\theta)$, CX , and $CR_y(\theta)$ which form a net controlled rotation by angle θ . It is flipped in-between the address qubit to change pixel $|0\rangle$ to pixel $|1\rangle$. At last, the measurement of both qubits is performed, and the process of FRQI encoding is finished. This can be used to demonstrate how quantum circuits can be used to model and manipulate classical image data in a quantum manner.

Decoodt *et al.* [49] showed the limitations faced by classical convolutional neural networks (CNNs) in natural image settings. Here, they considered a special case of histopathological image and demonstrated the challenges of extracting high-level features under constraints of limited data and computational power. The authors circumvented these bottlenecks by introducing a quantum-enhanced hybrid NN using the quantum machine learning (QML)

module of *Qiskit*. In particular, a ResNet50-based classical feature extractor was used to downsize the dimension of the image and the features obtained were subsequently fed to a variational quantum circuit implementation constructed using *Qiskit qiskit-machine-learning* library.

The quantum classifier was built around a data reuploading strategy. Here, the classical inputs like vectors $\vec{x} \in \mathbb{R}^d$ were initially encoded into quantum states using $R_y(x_i)$. Subsequently, they were processed into trainable unitary $U(\theta)$ to form a layered ansatz. The output of the circuit was measured on a selected qubit for generating prediction. The quantum layer was used to calculate a class score:

$$f(\vec{x}, \theta) = \langle 0 | U^\dagger(\theta) R_y^\dagger(\vec{x}) Z R_y(\vec{x}) U(\theta) | 0 \rangle, \quad (10)$$

where $f(\vec{x}, \theta)$ denotes the expectation value of the observable Z on the output qubit, with θ representing the trainable variational parameters. *Qiskit Estimator* was used to calculate the expectation during the training phase. The resulting hybrid model was then optimized using a classical optimizer and implemented on IBM Q-backends, including the `ibmq_manila` device.

The proposed hybrid quantum-classical network model achieved a significant result of 95.6% classification accuracy on the *BreakHis* dataset for binary breast cancer classification. This outperformed both the classical standalone CNNs and also the quantum models. These results show the potential of *Qiskit*-enabled hybrid architectures that can help in enhancing diagnostic performances even with constraints on limited data and resources.

Reka *et al.* [50] showed the limitations of ML algorithms while dealing with complex skin disease classifications. This is because of the intricate and high-dimensional nature of data involved in medical imaging. They used QML models to increase the efficacy of skin disease classification. Their study included two approaches using *Qiskit* and *PennyLane* frameworks. In Quvolutional NNs, quantum rotations are used to transform the local image patches. A 2×2 region encoded in a four-qubit quantum state is processed through a unitary operation U , with the expectation values being measured using *Pauli*-gates:

$$\langle \psi | P_i | \psi \rangle = \text{Tr}(\rho P_i), \quad (11)$$

where $|\psi\rangle$ is used to represent the quantum state following rotations r_x, r_y, r_z of angles scaled by π, ρ , P_i represent the Pauli operators (x, y, z) and Tr represents Trace. Each expectation value maps onto a channel in the output pixel, allowing us to replace convolution kernels of classical counterparts with quantum transformations.

In Quantum Support Vector Classifier (QSVC), features are often extracted from MobileNet, and are encoded using the `ZZ FeatureMap` to identify the pairwise correlations. The quantum kernel is computed as:

$$K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2, \quad (12)$$

where $\phi(x_i)$ transforms the classical data x_i into amplitudes of quantum states and K_{ij} is used in the feature space to measure the similarity between data points. On the one hand, they used the `default.qubit` device of *PennyLane* quantum device for quvolutional layers with different qubit rotations and Pauli measurements to learn. On the other hand, *Qiskit FidelityQuantumKernel* was used to generate quantum kernels for QSVC classification. This quantum architecture attains 82.86% accuracy with R_y rotation and *Pauli*Z-gates, while QSVC records 72.5 % accuracy on the HAM10000 dataset. These results are much better than those achieved using all-classical models.

3.2.2 Telecommunications

QC in telecommunications overcomes the drawbacks of a classical network viz. high spectral crowding, signal processing latency, and massive Maximum-Input Maximum-Output (MIMO) challenges. With new technologies like 5G and 6G architectures in place, there is a need for accurate channel estimation and beamforming. However, classical systems fail to perform these tasks due to discrete hardware and error propagation in noisy environments.

Quantum models like Quantum Orthogonal Frequency-Division Multiplexing (Q-OFDM) overcomes these limitations by executing multiple signal functions in parallel. This can be done using basic principles like entanglement, superposition and Quantum Fourier Transform (QFT) which significantly increase the channel robustness. *Qiskit* makes this process simpler by tracking performances with metrics like bit error rate (BER), latency, and spectral efficiency. Almasaoodi *et al.* [51] proposed Q-OFDM to reduce signal overlap, while Sabaawi *et al.* [52] demonstrated its reliability under rotation-based noise. Subsequent studies by Anjimon *et al.* [53] used quantum linear solvers like Harrow-Hassidim-Lloyd (HHL) algorithm to provide scalable approaches to channel estimation and beamforming, showing significant reductions in computational time. These developments highlight the potential for telecommunications use cases that can be solved using *Qiskit*.

$$|\psi_f\rangle = \text{IQFT} \cdot R_x(\theta)^n \cdot \text{QFT}|\psi_0\rangle, \quad (13)$$

where $|\psi_0\rangle$ denotes the initial quantum state, $R_x(\theta)^n$ applies rotation noise across n qubits, and $|\psi_f\rangle$ represents the final state after inverse transformation and decoherence. QFT and IQFT stands for quantum Fourier transform and inverse quantum Fourier transform, respectively.

In the code shown below, we simulate a 4-qubit Q-OFDM system using *Qiskit* (lines 1-18). The procedure starts with the initiation of 4-qubit register with classical bits to be measured (line 2). The circuit then uses a QFT to transform the input bitstring into the frequency domain by using `QFTGate(4)` (line 5). In order to simulate realistic phase noise, $R_x(50^\circ)$ -gates are applied on each qubit, which are the simulated effects of rotational distortions in the channel (lines 8-9). After this, *IQFT* is performed with `.inverse()` to decode the signal and return it back to the time domain (line 12). Lastly, the output state is measured on all qubits (line 15) and the circuit is plotted (line 18) to view its structure. The associated bitstring distribution shows a noise-affected Orthogonal Frequency Division Multiplexing (OFDM) signal, and the circuit represents the quantum analog of the encoding, distortion, and decoding in classical communication systems.

```

1 # Build 4-qubit circuit with classical bits for measurement
2 qc = QuantumCircuit(4, 4)
3
4 # Apply QFT
5 qc.append(QFTGate(4), range(4))
6
7 # Simulate noise
8 for q in range(4):
9     qc.rx(np.radians(50), q)
10
11 # Apply inverse QFT
12 qc.append(QFTGate(4).inverse(), range(4))
13
14 # Measurement
15 qc.measure(range(4), range(4))
16
17 # Draw the circuit
18 qc.draw('mpl')
```

Fig. 5 shows how QFT is used to encode signals in 4-qubit noisy quantum system. QFT is initially performed on the qubit q_0 to generate a superposition state in frequency domain. After that, a uniform noise operation is applied to the four qubits with rotations about $R_x(50^\circ)$. This models the phase noise that can exist on a real world communication channel. The circuit then acts on q_0 with IQFT attempting to obtain a decoded signal to see how the transformation fares against noise. The other qubits go through noise layer alone. In the end, each qubit is measured to examine the final state. It shows how quantum circuits implementing QFT can be used to simulate the modulation/remodulation steps needed to deliberate to the signal in a quantum-enhanced communication system (and how rotational noise degrades the fidelity and phase coherence).

Almasaoodi *et al.* [51] showed how Q-OFDM is used to maintain reliable communication in noisy conditions. They developed a cyclic prefix-free Q-OFDM circuit built on a 8-qubit quantum register. The data is encoded

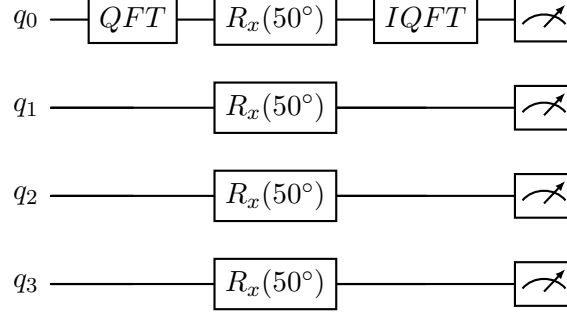


Figure 5: Quantum Circuit for a 4-qubit QFT system showing the effect of noise by applying $R_x(50^\circ)$ to each qubit.

using IQFT and decoded with QFT to approximate the phase noise by $R_x(50^\circ)$ rotations. *Qiskit* running on *AerSimulator* with more than 1024 shots resulted in a BER of 1.4%, exhibiting good phase noise immunity. This study showed how *Qiskit* can support the simulation and optimization of quantum communication protocols that are relevant in future 5G and 6G networks.

Sabaawi *et al.* [52] worked on Q-OFDM by investigating the effect of Pauli twirling to reduce the impact of noise. Here, they considered three noise models and measured the performance of Q-OFDM over an array of channel rotations. It was observed that the addition of Pauli twirling had reduced the BER by up to 45% with a 100° rotation, hence providing improved reliability in noisy channels. These findings confirm *Qiskit*'s ability to simulate quantum communication protocols when dealing with noise and highlights the possibility of quantum systems in providing excellent communication capabilities.

Anjimon *et al.* [53] proposed a variational quantum regressor (VQR) model to optimize signal routing in massive MIMO arrays. The authors construct a hybrid quantum-classical VQR framework using *Qiskit* that improves the optimization of beamforming weights while simultaneously reducing computational overhead. The proposed VQR-based method demonstrates a 92% reduction in runtime compared to classical solvers when solving for 512×512 matrix computations.

Zhang *et al.* [54] employed a hybrid quantum-classical approach aimed at optimizing beamforming strategies in 5G communication systems. The model was built using a quantum neural network (QNN) into a classical convolutional network to enhance the efficiency of downlink beamforming. Quantum circuit layers were then simulated in the hybrid architecture and demonstrated that the quantum-assisted network outperforms classical networks by achieving comparable sum-rate performance with up to ten times fewer trainable parameters. The loss function used for the optimization is:

$$L = -\frac{1}{2KN} \sum_{n=1}^N \sum_{k=1}^K \log_2 \left(1 + \gamma_k^{(n)} \right) \quad (14)$$

where K is the number of users, N is the batch size, and $\gamma_k^{(n)}$ is the signal-to-interference-plus-noise ratio (SINR) for user k in the n -th sample. This framework demonstrates the ability of *Qiskit* to combine QML methods involving resource allocation with the aim of improving beamforming performance in future wireless systems.

Urgelles *et al.* [55] showed how *Qiskit* can help in detection of MIMO-Non-Orthogonal Multiple Access (NOMA) systems for 6G networks. QAOA algorithm was used in a 2×2 BPSK modulation system. The QAOA-based detector was run on the IBM Quantum device with 1024 shots and results showed that BER was in general agreement with traditional ML detection. The QAOA cost Hamiltonian of this architecture is as follows:

$$H_P|x\rangle = \left(\sum_{i,j=1}^n x_i Q_{ij} x_j + \sum_{i=1}^n L_i x_i \right) |x\rangle \quad (15)$$

Here, H_P represents the Hamiltonian problem containing cost function and mixing problem H_m , H_m is defined as the sum of single Pauli X operators on all the qubits, x_i is the i -th binary variable representing a qubit state (0 or 1), Q_{ij} is the coefficient representing the quadratic interaction between variables x_i and x_j , L_i is the linear coefficient for variable x_i , i and j are summation indices, and n is the total number of variables (or qubits). These findings show that *Qiskit* provides a reliable framework using which quantum-based optimizations in MIMO detection can be achieved.

Rexhepi *et al.* [56] developed Quantum Manifold Optimization (QMO) which is a new paradigm that reformulates traditional wireless design problems (such as pilot allocation, massive MIMO beamforming, and reconfigurable intelligent surface phase tuning) as quantum expectation problems in terms of the trace operation. By designing Riemannian-constrained quantum circuits and simulating them with the help of *Qiskit*, they resolved all these problems. The method maps beamformer design to quantum manifolds, including Stiefel or Grassmann. Consequently, the method evaluates them through variational quantum algorithms. The beamforming objective is formulated as:

$$\min_W \text{Tr}(W^H A W), \quad \text{subject to} \quad W^H W = I \quad (16)$$

where W is the Beamforming matrix and A is the Hermitian matrix representing system or channel parameters. W^H shows the Hermitian transpose of W , I is the identity matrix, and the constraint $W^H W = I$ enforces orthonormal columns in W . The cost function of quantum Hamiltonian is applied to a variational quantum algorithm, which is used to minimize the expectation value. *Qiskit*-based simulations shown here validate that geometric formulations enable efficient optimization of complex wireless design tasks. These results highlight the future potential of *Qiskit* for addressing the requirements of the next-generation 5G/6G communication systems.

Siddiqui *et al.* [57] proposed a QNN-based on *Qiskit* to improve performance measures in wireless networks. Variational quantum circuits was used to build QNN layers and train the model using a mean-squared error loss function. This discovery shows that QNNs can work with performance accuracy comparable to that of traditional ones but with a much smaller number of trainable parameters. This paper supports the appropriateness of *Qiskit* to build hybrid quantum model in communication optimization.

Pathak *et al.* [58] showed how resource reallocation can be done in complex 5G networks using *Qiskit*. VQR aimed at optimizing resource allocation under the constraints of dynamic network conditions, and this model demonstrated an impressively low mean squared error of 0.0081 during simulations implying its perfect predictive ability. Quantum regression (especially with variational circuits) is a practical alternative to classical ML in applications where latency is a primary concern and throughput demands are high including in 5G networks. The paper effectively highlights that *Qiskit* can serve as a simulator in the implementation of quantum algorithms for telecommunications.

3.3 Climate Science and Energy Applications

High performance computers are commonly used for classical climate modelling and energy simulations. They are used to solve differential equations and grid configurations using Monte Carlo simulation methods and numerical optimization tools. The capabilities often fall short, since these methods face barriers during high energy demands and reduced fidelity when dealing with complex, nonlinear high-dimensional systems like global climates or vast energy grids [59]. The potential of new technological solutions employing QC to overcome these limitations is also interesting. QC uses quantum parallelism and entanglement to perform sampling, optimization and simulation tasks that are impossible to perform with a classical computer [60]. *Qiskit*, especially, has been playing a key role in this transition of paradigm. The platform has ready modules, *Qiskit Nature* and *Qiskit Optimization* to apply quantum enhancements in energy systems modeling, and variational methods that apply to climate science and renewable-energy forecasting [61]. This discussion presents the possibility of using *Qiskit* to guide researches to advance sustainable technology and environmental models using quantum-enabled processes.

3.3.1 Climate Modeling and Simulation

Numerical methods like finite difference, finite element and Markov Chain Monte Carlo (MCMC) are used in classical climate modeling. These methods can solve partial differential equations and can sample transition odds between discretized states of the atmosphere or ocean. However, they possess significant processing needs when dealing with occurrences of extreme events that are not frequent [62]. QC, with the help of features like amplitude amplification and superposition, yields quadratic time advantages in sampling and in solving differential equations [63].

The code shown below demonstrates a Quantum Phase Estimation (QPE) algorithm that estimates the inverse of small, discretized linear operators that can be applied directly to climatic systems. In this respect, we build a matrix A and a vector b (lines 1-4). Then, we create a 3-qubit circuit (line 7) and initialize the initial state $|b\rangle$ (lines 10-11). A controlled phase QPE is used (lines 14-16) and IQFT is applied (lines 19-21). These inverse eigenvalues are then encoded using a controlled rotation circuit on the system qubit (lines 24-25). We then measure the ancilla qubits after uncomputing QPE (lines 28-29). After measuring these states in the system qubit, we will get an estimate of the solution $x = A^{-1}b$. This deterministic implementation avoids the use of high-level solvers and shows how discrete PDEs can be inverted using quantum circuits to accelerate climate modelling using quantum hardware.

```
1 # define A, b; compute eigenpairs
2 A = np.array([[1.0, 0.5], [0.5, 1.0]])
3 b = np.array([1.0, 0.0])
4 eigvals, eigvecs = np.linalg.eigh(A)
5
6 # initialize 3-qubit circuit (1 system + 2 ancilla)
7 qc = QuantumCircuit(3, 2)
8
9 # prepare |b> via Ry rotation
10 theta = np.arccos(b[0] / np.linalg.norm(b))
11 qc.ry(2 * theta, 0)
12
13 # QPE to encode eigenvalues on ancilla
14 for j in range(2):
15     qc.h(1 + j)
16     qc.cp(2 * np.pi * eigvals[j], 1 + j, 0)
17
18 # inverse QFT on ancilla qubits
19 qc.h(2)
20 qc.cp(-np.pi/2, 1, 2)
21 qc.h(1)
22
23 # controlled rotations to encode 1/lambda on system qubit
24 qc.crz(np.pi / eigvals[0], 1, 0)
25 qc.crz(np.pi / eigvals[1], 2, 0)
26
27 # uncompute QPE and measure ancilla
28 qc.h(1); qc.cp(np.pi/2, 1, 2); qc.h(2)
29 qc.measure([1, 2], [0, 1])
```

The circuit in Fig. 6 implements QPE for solving a 2×2 linear system of equations. The initial state of system qubit is set to the state vector $|b\rangle$, and is encoded by a single operation of rotation $R_y(2\theta)$ to achieve the desired input state. There are two ancilla qubits initialized in the $|0\rangle$ state and their superposition is created with the H -gates. These ancilla qubits are then entangled with the system qubit by some controlled unitary operations, which allow encoding of eigenvalues of matrix A as phases of ancilla states. This is followed by an IQFT through H -gates and controlled-phase, enabling the circuit to gather phase (eigenvalue) information. The last step is to take measurements on the ancilla qubits to obtain the result, and the system qubit which has retained the entanglement

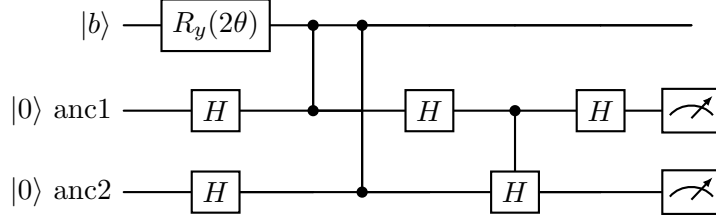


Figure 6: QPE-based 2×2 quantum circuit showing a linear system solver when using controlled phase encoding and IQFT.

is known as the encoded solution state. The quantum circuit can thus implement quantum linear solvers including the HHL algorithm, demonstrating the usefulness of QPE to model matrix inversions.

Choudhury *et al.* [64] suggested using *Climate Deep Long Short-Term Memory (CDLSTM)* classical model trained on Himalayan temperature data (1796 to 2013) and rainfall data (1901 to 2015). This model was benchmarked against Facebook-owned ML platform named Powell Prophet, and realized a roughly 15% reduction in root mean-squared error (RMSE). This is due to accuracy of the gating mechanism present in the Long Short-Term Memory (LSTM) model that, which allows for capturing more complex and long-ranged dependencies of sequences. These are particularly crucial in understanding the dynamics of climatic patterns.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - O_i)^2} \quad (17)$$

where Y_i denotes the i -th predicted value of LSTM, O_i shows the i -th original value, and n is the total number of samples.

Sebastianelli *et al.* [65] showed a hybrid framework that classifies images for remote sensing. This paper used a novel quantum convolutional neural networks (QCNN)-based method that was tested on the EuroSAT dataset for benchmarking results. They used a classical CNN architecture of **LeNet5** design along with additional quantum layer stacked between the classical layers. With an F1 score and accuracy of 92%, this hybrid model significantly surpasses both pure classical and pure quantum methods. The overall equation of a 4 qubit system as shown in the paper can be represented as:

$$U^* = \bigotimes_{i=0}^{i=3} (R_y(\theta_i) \cdot H) \quad (18)$$

where U^* is the unitary transformation, and R_y and H are rotation gate and Hadamard gate respectively. Now, the unitary transformation is performed on the tensor product ($\otimes$) of all four qubits:

$$|\psi\rangle = U^*(|\psi_0\rangle \otimes |\psi_1\rangle \otimes |\psi_2\rangle \otimes |\psi_3\rangle) \quad (19)$$

Bhavsar *et al.* [66] discussed how potentially hazardous asteroids impact biodiversity ranging from seismic shaking, overpressure shock, tsunamis etc. They introduced a supervised-QML approach for asteroid hazard prediction using Variational Quantum Classifier (VQC) algorithm. The proposed approach of using VQC and PEGASOS Quantum Support Vector Classifier (PegasosQSVC) significantly outperformed other models and showed an accuracy of 98.11% and an F1 score of 92.69%. PegasosQSVC can be outlined from the basis of Pegasos Quantum Support Vector Machines (QSVM) algorithm as shown in the equation below:

$$\mathcal{O}_{\text{pegasos}} = \min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^m \max\left(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i\right)^2 \quad (20)$$

where O_{pegasos} shows the regularization term of the optimization problem, $\mathbf{w}$ is used to show the weight vector, C is regularization parameter, and x_i and y_i denote the input vector and its corresponding label respectively. Their optimization problem then found the weight vector that minimized the objective function.

Danyal *et al.* [67] conducted a study on the use of QC to build surrogate models. The authors of this study tried forecasting the temperature and spatial position of heat plumes in Munich. Due to the current limitations in quantum hardware and software, the initial data was simplified and truncated to low-dimensional input-output space. They used the QCNN model employing quantum circuits to simulate the classical CNN convolution and pooling functions. The QCNN variant was run on `ibmq`, and *Qiskit* state vector and fake back-end were used to simulate the hardware potential defects while integrating IBM noise models. The mean-squared error (MSE) was used to determine the model performance.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (21)$$

where y_i shows the true values of simulation and $\hat{y}_i$ shows the predicted values. The results indicate that quantum models are better under low MSE and noisy conditions. This helped support the claim of feasibility of quantum-enhanced learning for physical surrogate modeling with quantum devices.

Gupta *et al.* [68] explored the use of QML for classification of multispectral earth observation data that was gathered in Copernicus Sentinel-2 mission. Projected Quantum Kernel (PQK) approach was introduced in this study which was developed to complement classical approaches, including Support Vector Machines (SVMs) and decision trees. Results showed that there was a dramatic rise in the accuracy of classification with the addition of PQK features. The overall performance was much better in comparison to all-classical methods. The game-changing metric is the decision due to the quantum model:

$$f(x) = \sum_{i=1}^N \alpha_i K_{\text{PQK}}(x, x_i) + b, \quad (22)$$

where $K_{\text{PQK}}(x, x_i)$ shows how quantum projected kernel can help map the multispectral pixel vectors x and x_i into a higher-dimensional Hilbert space. Here α_i is the weight introduced during training and b shows the bias term. By using this embedding architecture, the work captured the nuances of the spectrum not captured in the traditional kernel-based approaches and showing the ability of QML to increase the capacity of processing of the Earth observation data.

This operation $\text{Tr}_{j \neq k}$ where $j \neq k$ can be shown as:

$$\text{Tr}_{j \neq k}[\rho(x_i)] = \sum_{j \in \{0,1\}^{k-1}} \sum_{j' \in \{0,1\}^{D-k+1}} \text{Tr}[(P_j \otimes I \otimes P_{j'}) \rho(x_i)], \quad (23)$$

where $\rho(x_i)$ shows projected quantum kernel, I is identity operator, $P_j = |j\rangle\langle j|$ and $P_{j'} = |j'\rangle\langle j'|$ are projection operators.

Munasinghe *et al.* [69] benchmarked the applicability of QML methods specifically for climate action tasks. They mainly focused on weather and climate data that was taken and processed from NASA satellites. The study in general uses two different QML models viz. VQC and QSVC. Both of these were used to benchmark against classical ML approaches. The following experiments were carried out on IBM's 127 qubit quantum computer (Eagle) along with the *Qiskit* ML framework. To benchmark against other models, a special parametrized cost function was used:

$$L(\theta) = - \sum_i [y_i \log P(0 | x_i, \theta) + (1 - y_i) \log(1 - P(0 | x_i, \theta))] \quad (24)$$

where θ is the trainable parameter, $L(\theta)$ is the loss function, y_i is the true label, and $P(0 | x_i, \theta)$ denotes the probability of measuring a particular outcome.

3.3.2 Energy Optimization and Smart Grids

Classical optimization methods like Mixed-Integer Linear Programming (MILP) and heuristic algorithms have been applied to find smart grid solutions over the years by finding a balance between generation constraints and load constraints [70]. These methods have severe shortcomings when dealing with smart grids that are described by high shares of renewable energy sources, decentralized energy production, and variable load profiles. MILP-based models are computationally expensive and might not perform effectively on large-scale real-time energy scheduling problems [71]. QAOA provides a hybrid quantum-classical architecture that has the potential to overcome the limitations of all-classical models by comprehensively sampling large solutions spaces [72]. *Qiskit* also offers the means to model energy systems as graphs and deploy quantum gates to optimize processes via hybrid quantum-classical computing. By running such algorithms on simulators and hardware, one can explore much larger solution spaces in comparison to classical approaches. This results in more robust and renewable energy systems.

Since modern smart grids have complex structures and dynamics, we need to accurately represent the system as weighted graph (lines 2-6). Here, the substations are represented as nodes and the transmission lines which are characterized by energy loss can be represented as edges. We try to mirror the networked architecture of real-world energy systems, with transmission inefficiency being a critical factor. To minimize the energy losses, we set up quadratic unconstrained binary optimization (QUBO) matrix (lines 9-16). This is to capture the objective of controlling aggregate transmission losses over the network by deciding on operating connection in binary optimum manner. We then encode the QUBO matrix into *QuadraticProgram()* function, and prepare it for the quantum optimization problem (lines 19-23). QAOA with a classical Constrained Optimization by Linear Approximation (COBYLA) optimizer is used to adjust parameters (lines 26-27) and is run on a simulated quantum backend to emulate the behavior of available near-term quantum hardware (lines 30-31). The *MinimumEigenOptimizer* converts the quantum output into intelligible answers (line 34). The final solution corresponds to an optimal energy distribution strategy with equivalent minimized loss (lines 37-38). Using the quantum-augmented optimization as provided by QAOA, the model tackles the combinatorial nature of energy management in smart grids, while the conventional solvers suffer from scalability issues. This code provides a framework for producing clean and affordable energy (UN SDG 7) using QC.

```
1 # 1. Represent the smart grid as a graph
2 graph = nx.Graph()
3 graph.add_weighted_edges_from([
4     (0, 1, 3), (0, 2, 2), (1, 2, 1),
5     (1, 3, 4), (2, 3, 5)
6 ])
7
8 # 2. Create a QUBO matrix to minimize energy loss
9 Q = {}
10 for i in graph.nodes:
11     for j in graph.nodes:
12         if i == j:
13             Q[(i, i)] = 0
14         elif graph.has_edge(i, j):
15             loss = graph[i][j]['weight']
16             Q[(i, j)] = loss
17
18 # 3. Set up the Quadratic Program
19 qp = QuadraticProgram()
20 for i in graph.nodes:
21     qp.binary_var(name=f'x{i}')
22 for (i, j), w in Q.items():
23     qp.minimize(quadratic={('x' + str(i), 'x' + str(j)): w})
24
25 # 4. Setup QAOA optimizer
```

```

26 optimizer = COBYLA()
27 qaoa = QAOA(optimizer=optimizer, reps=1)
28
29 # 5. Backend selection
30 backend = Aer.get_backend('qasm_simulator')
31 quantum_instance = QuantumInstance(backend, shots=1024)
32
33 # 6. Build Minimum Eigen Optimizer
34 meo = MinimumEigenOptimizer(qaoa.run(quantum_instance))
35
36 # 7. Solve the problem
37 result = meo.solve(qp)
38 print(result)

```

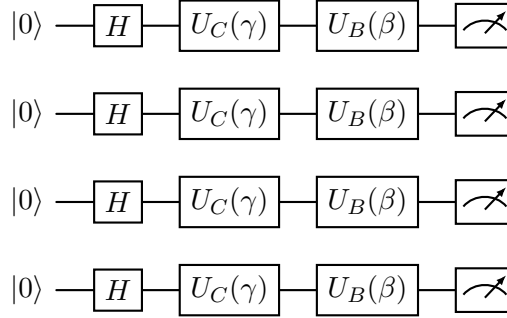


Figure 7: Quantum circuit of QAOA optimization problem where smart grids are modeled as weighted graphs

Fig. 7 gives an example of QAOA circuit applied to the solution of optimization problems in smart grid networks. The system starts with the initialization of four qubits in the $|0\rangle$ state, then H -gates are applied to produce equal superposition of all the possible configurations, which correspond to different energy routing paths. The unitary $U_C(\gamma)$ uses a graph cluster problem cost Hamiltonian that is a weighted graph model of the grid in which the weight of the edge corresponds to an energy loss or transmission expense. This is then succeeded by a mixing unitary $U_B(\beta)$ that searches within a set of other states so that feasibility of the solution remains. The ordering of cost and mixing unitaries optimizes ground state that gives minimum energy loss throughout the grid. Measurements are made after a series of such layers to determine the best configurations – for simplicity, only one layer is shown in the figure. Circuit-based quantum algorithms including combinatorial optimization thus provide a strong framework for quantum-enhanced energy systems.

Tang *et al.* [73] used a secure network-based quantum microgrid architecture using QKD for microgrid communications to resist attacks from quantum adversaries. They used a BB84 QKD scheme in decoy state, and designed a Key-Pool Sharing (KPS) strategy to develop denial-of-service architecture for reducing excessive consumption on one link. To ensure resilience, they introduced Two-Level KPS (TLKPS) to redistribute keys (K_{ij}) below thresholds T through intermediate sharing layers as shown below:

$$\text{if } K_{ij} < T, \quad K_{ij} \leftarrow K_{ij} + \sum_{k \neq i,j} \alpha_{ik} K_{ik}, \quad (25)$$

where α_{ik} denotes a sharing coefficient. The experiments showed that, in the single-attack scenario, K_{AC} experiences a critical bit shortage. In TLKPS, if K_{AC} drops below 10,000-bit threshold, 50,000 bits are transferred from K_{AB} to K_{AC} . Here, K_{AC} and K_{AB} are the key pools shared between microgrids A and C , and between microgrids A and B , respectively. However, in the multi-attack scenario, TLKPS effectively mitigates the bit shortage in K_{AC} by dynamically sharing 50,000 bits from either K_{AA} or K_{CC} , demonstrating robust key recovery even under multiple simultaneous quantum channel attacks. Here, K_{AA} and K_{AC} are the key pools held locally at microgrids A and C respectively.

Zhou and Zhang [74] developed a noise resilient QML framework that emphasizes on transient stability assessment in power systems using QNNs. This starts with encoding classical input data x , such as generator and voltage bus states, into quantum states $|\psi(x)\rangle$ such that they are passed through parameterized unitary circuits $U(\theta)$. The final output is obtained by measuring an observable O with the prediction modeled as:

$$y = \langle \psi(x) | U(\theta) O U(\theta) | \psi(x) \rangle. \quad (26)$$

To further optimize the circuit parameters θ , they used a quantum natural gradient, wherein a standard gradient is applied by incorporating the geometry of the quantum state space using the Fisher information matrix $\mathcal{F}$:

$$\Delta\theta = -\eta \mathcal{F}^{-1} \nabla_{\theta} \mathcal{L}, \quad (27)$$

where η is the learning rate and $\mathcal{L}$ is the loss function. This technique is used to enhance convergence and stability while training the model.

Koretksy *et al.* [75] transformed the unit commitment problem which was formulated into binary variables $y_i \in \{0, 1\}$ showing whether the generator i is activated. The cost for unit i is shown by the equation:

$$H_i(y_i, p_i) = A_i y_i + B_i p_i + C_i p_i^2, \quad (28)$$

with the constraints being $p_{\min, i} y_i \leq p_i \leq p_{\max, i} y_i$ and the total load $\sum_i p_i = L$. Here $A_i, B_i, C_i \in \mathbb{R}$ are constants, and L is the total load. QAOA is then applied with depth p , alternating between U_C and U_M :

$$U_C(\gamma) = e^{-i\gamma H_{QUBO}}, \quad U_M(\beta) = e^{-i\beta \sum_j X_j}, \quad (29)$$

The angles (γ, β) are optimized using a classical outer loop. Here, H_{QUBO} is the QUBO Hamiltonian and X_j is the quantum NOT gate applied on j -th qubit. *Qiskit* simulations proved that, for the systems with >400 units, QAOA maintains a stable performance, while classical solvers demonstrate exponential scaling,

Han *et al.* [76] proposed a novel approach for Monte Carlo optimization algorithm using multi-layered parallel QC to optimize control parameters K_p and K_i of rotor-side converters in doubly-fed induction generators (DFIG). The algorithm model helps optimization process by combining the quantum probability amplitudes of qubits using the Monte Carlo sampling which helps in exploring the search space globally. The qubit amplitudes of the update rule α_j, β_j in the j -th layer is given by:

$$\begin{pmatrix} \alpha'_j \\ \beta'_j \end{pmatrix} = \begin{pmatrix} \cos(\theta_j) & -\sin(\theta_j) \\ \sin(\theta_j) & \cos(\theta_j) \end{pmatrix} \begin{pmatrix} \alpha_j \\ \beta_j \end{pmatrix}, \quad (30)$$

in which θ_j is the rotation angle based on the Monte Carlo random number series and directed by objective-function analysis of candidate solutions. This process regulates the state path to parameters that help maximize power delivery and efficiency. This multi-parallel layered algorithm was shown as a quantum-assisted iterative optimization algorithm. It proves to converge more quickly and precisely to the theoretical global optimum compared to classical optimization techniques.

Ajagekar *et al.* [77] presented a quantum-classical hybrid deep learning framework to detect the faults in electrical power systems. Their approach used a classical CNN along with a parametrized quantum circuit to use quantum feature encoding. The input feature vector $\mathbf{x}$ is initially processed classically, and then embedded into quantum state using an encoding unitary $U(\mathbf{x})$ such that:

$$|\psi(\mathbf{x})\rangle = U(\mathbf{x}) |0\rangle^{\otimes n}, \quad (31)$$

where n denotes number of qubits, and $|0\rangle^{\otimes n}$ is the initial quantum state of n qubits initialized to the computational basis state $|0\rangle$. The quantum circuit is parameterized by training angles θ , producing the quantum state:

$$|\psi(\mathbf{x}, \theta)\rangle = U(\theta) U(\mathbf{x}) |0\rangle^{\otimes n}. \quad (32)$$

The hybrid framework enables a quantum enhanced feature representation that is used to improve the accuracy of fault detection. Experiments were performed on power systems fault datasets that showed a superior performance of quantum models compared to classical deep learning models.

It is also interesting to note that some of the most prominent companies have started using QC for providing clean and affordable energy (UN SDG 7). E.ON has collaborated with IBM Quantum [78] to promote the energy transition, grid optimization and energy efficiency using quantum-powered simulations and optimization processes. A Tennessee-based startup applies quantum technologies to optimize micro-siting for wind turbines, allowing for more energy efficient placements adapted to complex terrain and atmospheric conditions [79]. IonQ and Hyundai Motor are working on the use of QC in battery chemistry exploration to manufacture next-generation, efficient, and sustainable electric-vehicle batteries [80].

3.4 Finance and Risk Analysis

Today’s financial systems, which form the backbone of global markets and economic stability, are confronted with escalating challenges from increasing market volatility, complex optimization demands, and the need for real-time risk assessment [81]. *Qiskit* tackles these challenges by making possible the development of quantum-enhanced models for PO and risk mitigation [82]. This section examines how *Qiskit* bridges quantum algorithmic theory with financial workflows, allowing institutions to tackle computationally demanding problems while working within the constraints of today’s quantum hardware.

3.4.1 Portfolio Optimization

PO seeks to maximize returns while minimizing risk across a given set of assets, and it has evolved into a fundamental element of modern investment strategies. Traditional methods, like Markowitz’s mean-variance approach [83], run into serious computational roadblocks when dealing with large portfolios as the complexity scales with asset correlations and constraints [84]. QC changes the game by encoding portfolio weights into quantum states and make use of superposition and entanglement to efficiently explore optimal allocations.

QAOA has emerged as one of the most promising methods for this application, mainly because it works well with today’s quantum hardware. It translates financial objectives into a parameterized quantum circuit, then gradually improves solutions through hybrid quantum-classical feedback loops. *Qiskit*’s `PortfolioOptimization` module [85] converts the optimization problem into a QUBO model [86], where the choice of assets and allocating weights are represented as qubit states.

The following code snippet illustrates how a portfolio with transaction cost constraints is encoded into a QAOA circuit using *Qiskit*’s Finance library to define risk-return tradeoffs and execute the optimization workflow on quantum hardware.

```

1 # Define assets, returns, and covariance matrix
2 num_assets = 3
3 returns = np.array([0.1, 0.2, 0.15])
4 covariance = np.eye(num_assets) * 0.1
5
6 # Encode portfolio problem as QUBO
7 portfolio = PortfolioOptimization(returns, covariance, risk_factor=0.5, budget=2)
8 qubo = portfolio.to_quadratic_program()
9
10 # Configure QAOA
11 qaoa = QAOA(reps=2, quantum_instance=Aer.get_backend('qasm_simulator'))
12 result = qaoa.compute_minimum_eigenvalue(qubo.to_ising()[0])
13
14 # Extract optimal asset selection
15 optimal_solution = portfolio.interpret(result)

```

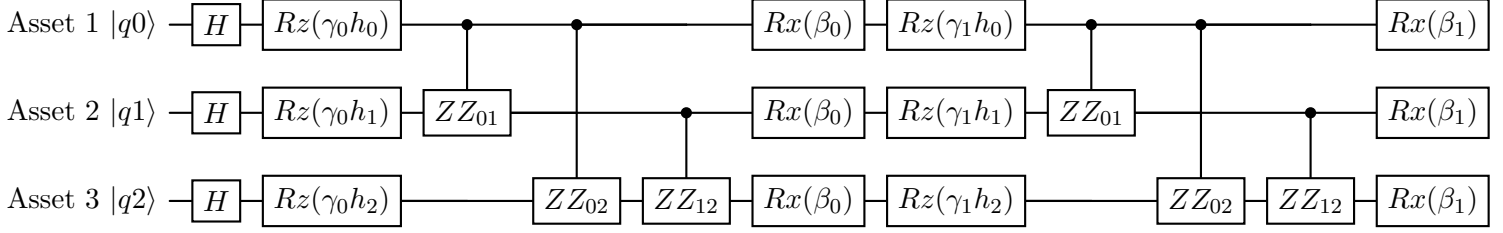


Figure 8: QAOA circuit representing a PO instance with three assets. The gates between $R_z(\gamma_0 h_0)$ and ZZ_{12} correspond to the problem Hamiltonian, while the surrounding $R_x(\beta)$ gates represent the mixer Hamiltonian. These two sets of operations are repeated for two iterations, illustrating layered application of both Hamiltonians.

In the beginning, we construct a fundamental financial model using three assets, each with distinct expected returns and uncorrelated risks (which explains our use of an identity matrix scaled by 0.1). In line 7, *Qiskit*'s `PortfolioOptimization` class takes all financial data and turns them into an optimization problem that has constraints. Here, we target the balance between good returns and low risk, and the `risk_factor` parameter lets us decide this. At the same time, it ensures that we are within the constraints of picking exactly two assets and staying within our budget. Line 8 converts classical optimization parameters into *Qiskit*'s standard `QuadraticProgram` format. In line 11, we construct QAOA with two repetitions (`reps=2`) i.e. two layers of alternating cost and mixer operators and ran it on `qasm_simulator`. In line 12, we input the Ising version of our QUBO to `compute_minimum_eigenvalue`, which allows QAOA to search for the lowest energy solution that corresponds to our optimal portfolio. Then in line 15, we use the `interpret()` method to decode the quantum result in the form of a bitstring that allow us to select our final asset list. Here, we need not explicitly code the QUBO or QAOA logic since *Qiskit* handles both the formulations and quantum-classical optimization steps internally.

The quantum circuit that *Qiskit* generates from this code takes care of all the complexity involved in manually designing quantum algorithms for PO. As evident from Fig. 8, 3 qubits (each representing an asset) go through a structured sequence of operations in multiple QAOA layers (`reps=2`), with each layer combines a problem Hamiltonian encoding our financial objectives and a mixer Hamiltonian exploring the solution space.

We start by applying H -gates across all the qubits, and this produces a superposition that contains every possible asset combination from $|000\rangle$ through $|111\rangle$. It covers all the potential portfolio configurations, where $|1\rangle$ and $|0\rangle$ indicate inclusion and exclusion of an asset respectively. QUBO model provides the mathematical foundation for quantum optimization. When dealing with PO, the QUBO formulation brings together returns, risk, and budget constraints into a single cost function:

$$\text{QUBO} = - \sum_i \mu_i x_i + \lambda \sum_{i,j} \sigma_{ij} x_i x_j + \rho \left(\sum_i x_i - B \right)^2 \quad (33)$$

In this setup, $x_i \in \{0,1\}$ indicates whether the i -th asset is included ($x_i = 1$) or excluded ($x_i = 0$), μ_i gives the expected returns, σ_{ij} represents the covariance between assets i and j , λ controls the level of risk tolerance, ρ penalizes any deviation from the budget constraint, and $B = 2$ ensures that exactly two assets are selected in the portfolio. *Qiskit*'s `PortfolioOptimization` class is convenient in this regard because it automatically does the translation, without the need for manually conversion of financial parameters into a QUBO instance.

After this, we convert QUBO into an Ising Hamiltonian (H_c) that quantum hardware can actually handle. We accomplish this conversion using the transformation $x_i \rightarrow \frac{1-Z_i}{2}$, where Z_i is the Pauli-Z operator acting on qubit i . This thus gives the Hamiltonian:

$$H_c = \sum_i \mu_i Z_i + \sum_{i,j} \sigma_{ij} Z_i Z_j + \lambda \left(\sum_i Z_i - 2 \right)^2. \quad (34)$$

In practice, when we examine the circuit, this Hamiltonian appears as parameterized $R_z(\gamma)$ rotations that adjust qubit phases based on μ_i and σ_{ij} , together with ZZ -interaction gates that create entanglement between qubits to represent the covariance relationship. The γ parameters (such as γ_0 , γ_1 , etc.) determine the strength of these operations throughout both layers of QAOA.

The mixer unitary (U_M), implemented via $R_X(2\beta)$ rotations, drives transitions between computational basis states, ensuring exploration of diverse portfolios. Each QAOA layer alternates between the cost unitary U_C and mixer unitary U_M , iteratively refining the quantum state towards low-energy (high-quality) solutions. With `reps=2`, the circuit runs through two of these layers, which gives us double the parameterized operations. When we do the measurement, the quantum state collapses and what we retrieve is a 3-bit string (could be `|110` for example). This string shows us a portfolio that is nearly optimal – one that successfully maximizes returns, minimizes risk, and meets our budget constraints. What this result tells us is exactly which assets we should include in our final portfolio selection. In the backend, the QAOA class takes care of the classical optimization loop automatically, working through systematic adjustments to the γ and β parameters so as to effectively minimize the portfolio’s cost function.

There has been some interesting work lately in quantum finance where researchers are figuring out how to use *Qiskit* for those really tricky optimization problems in finance, like managing portfolios, figuring out risk, and working out arbitrage opportunities. What they’re doing is taking *Qiskit*’s quantum-classical hybrid approach and using it to model and optimize all the different financial decisions to be made.

Carrascal *et al.* [87] used *Qiskit* as their main computational framework to encode and solve the cryptocurrency arbitrage problem. Their approach involved formulating the arbitrage optimization as a QUBO model using *Qiskit*’s Optimization module (*IBM CPLEX/Docplex engine*) to start with. Thereupon, the resulting binary quadratic model gets automatically translated into an equivalent Ising Hamiltonian. A parameterized variational Ansatz circuit (for instance, a layered **RealAmplitudesAnsatz**) was constructed in *Qiskit*. The VQE then attempts to minimize the expected energy $\langle \psi(\theta) | H | \psi(\theta) \rangle$ of that Hamiltonian by adjusting the variational parameters θ . By the variational principle:

$$\langle \psi(\theta) | H | \psi(\theta) \rangle \geq \lambda_{\min}, \quad (35)$$

where $\lambda_{\min}$ is the true ground-state energy. By driving this expectation towards its lower bound, we land up at the optimal arbitrage solution.

Instead of using *Qiskit*’s built-in optimizers, the authors integrated a custom Differential Evolution (DE) classical optimizer (implemented via SciPy) into the VQE loop. The hybrid quantum–classical routine was executed via the *Qiskit* Runtime environment. The *Qiskit* Runtime **Estimator** primitive is used to batch process expectation values for a population of ansatz parameter sets, and the *Qiskit* Runtime **Sampler** is used to evaluate the final converged state. This DE-driven VQE approach was validated on IBM Quantum hardware (for instance, the `ibm_geneva` processor), where it successfully converged to the ground-state arbitrage solution after 417 iterations for the three-currency test case.

Buonaiuto *et al.* [88] tackled the PO problem by using *Qiskit*’s end-to-end quantum workflow. They started by transforming the classical constrained quadratic optimization model into a QUBO formulation, which involved binary encoding of asset variables and adding penalty terms to enforce constraints. After that, QUBO was mapped to an equivalent Ising Hamiltonian, where the ground-state energy represented the optimal investment allocation. Using *Qiskit*’s variational algorithm tools, the authors implemented parameterized quantum circuits; specifically, the **TwoLocal**, **RealAmplitudes**, and **PauliTwo** ansatz, each defining different structures of variational states. The VQE is implemented with classical optimizers available in *Qiskit*, including COBYLA, Simultaneous Perturbation

Stochastic Approximation (SPSA), and Nakanishi Fujii Todo (NFT), to minimize the Hamiltonian’s expected energy. The VQE objective is expressed by the equation:

$$E_{\min} = \min_{\theta} \sum_{l=1}^D c_l \langle 0 | U^\dagger(\theta) P_l U(\theta) | 0 \rangle, \quad (36)$$

where $U(\theta)$ is the ansatz, θ is the set of variational parameters to be optimized, P_l are Pauli strings in the Hamiltonian decomposition, c_l are the real-valued coefficients associated with each Pauli term, and D is the total number of terms in the Hamiltonian decomposition. This *Qiskit*-based approach, from classical encoding to quantum optimization, was tested using both noiseless and noise-modeled simulators and real IBM NISQ devices, demonstrating effective convergence towards optimal portfolios for small asset sets.

In Quantum Finance Optimization Qubit (QFOQ) implementation [89], the researchers took advantage of *Qiskit*’s circuit libraries, specifically the `AmplitudeAmplification` class, to build Grover-style amplitude amplification routines. These routines increase the chances of selecting high-value financial signals. The team also conducted quantum Monte Carlo simulations by repeatedly sampling circuits with *Qiskit*’s built-in tools, providing a better way to forward-test financial models. To capture hardware-level imperfections, the authors used *Qiskit Aer*’s noise simulation capabilities, particularly the `NoiseModel` class, to emulate decoherence and gate errors in noisy financial environments. Scenario-based financial simulations and PO tasks were also executed using *Qiskit*’s primitives like *Sampler* and its variational solvers. The overall computational cost of the QFOQ pipeline is modeled by:

$$E_{\text{QFOQ}} = \alpha \cdot n \cdot T + \beta, \quad (37)$$

where n represents the number of financial variables, T represents the backtest duration, α shows the reduction coefficient for data dimensionality, and β accounts for fixed system overhead. This formulation demonstrated how *Qiskit*-based quantum circuits support scalable financial model testing through QFOQs.

Tripathy *et al.* [90] created a *Qiskit*-based pipeline to tackle the PO problem using variational quantum algorithms. They formulated the problem through *Qiskit* Finance modules, where they first convert the classical Markowitz mean–variance model into a quadratic program and later into a QUBO format that quantum hardware can handle. Since they were only able to handle 12-qubits, they split 48 assets into four buckets based on volatility and worked on each bucket independently. For each group, they used VQE and QAOA with a `RealAmplitudes` ansatz and the Sequential Least Squares Programming optimizer (SLSQP) to identify the best asset subset. This approach lets them construct a 16-asset portfolio by taking the top-performing assets from each bucket. The quantum solutions from this *Qiskit* pipeline performed as good as classical methods, but at significantly faster pace.

Yalovetzky *et al.* [91] implemented NISQ-HHL entirely in *Qiskit* by constructing custom circuits for the Quantum Circuit Learning (QCL)-enhanced phase estimation and inversion routines, and simulating them using *Qiskit Aer*. In particular, the QCL-QPE circuit in *Qiskit* exploits mid-circuit measurement, qubit reset, and classical conditional logic so that only one ancilla qubit is needed. In these *Qiskit* circuits, all multi-qubit controlled rotations are replaced by single-qubit gates conditioned on classical bits, yielding dramatically shallower depth and fewer two-qubit gates. The authors used *Aer*’s QASM simulator to validate eigenvalue estimation, including *Qiskit*-based scaling of the matrix A and classical post-processing of the QCL-QPE output distribution. Similarly, *Aer*’s statevector simulator ran the full pipeline, prepared the normalized portfolio state $|b\rangle$ and a reference $|x_c\rangle$, performed the eigenvalue inversion rotations, and executed a controlled-SWAP test between $|x\rangle$ and $|x_c\rangle$ to compute the output fidelity. The *Qiskit* simulations showed significantly higher fidelity than the standard HHL method. For instance, the inner-product overlap reached approximately 0.83 (simulated) versus 0.59 for the previous uniformly controlled rotation approach, and fidelities were essentially unity for larger portfolios. In summary, the *Qiskit* implementation demonstrated that QCL-QPE drastically reduced ancilla and depth requirements while achieving more accurate portfolio-state preparation on NISQ devices.

Using *Qiskit*’s modular architecture, Carrascal *et al.* [92] explored and benchmarked several quantum algorithms for PO using *Qiskit*, with a focus on VQE, Conditional Value at Risk-VQE (CVaR-VQE), QAOA, and Grover

Adaptive Search. The PO problem was encoded into a QUBO model, and the *Qiskit* library was used to transform it into a corresponding Ising Hamiltonian suitable for variational algorithms. The authors utilized real historical data from the *IBEX35* index and applied *Qiskit* simulators for extensive backtesting, while real IBM quantum backends (like Falcon and Eagle processors) were used for selected validation runs. VQE circuits were implemented using parameterized ansatz and optimized with *COBYLA* for its robustness and efficiency. CVaR-VQE, also realized in *Qiskit*, improved performance under noise by focusing on the lower-energy tail of sampled values [93]. QAOA was tested at varying depths but showed higher sensitivity to noise due to deeper circuits, while Grover Adaptive Search was explored only in simulation given its suitability for future fault-tolerant devices. The *Qiskit Session* feature was employed to efficiently manage repeated circuit executions on real devices, mitigating the latency caused by queue times. Experimental results showed that VQE and CVaR-VQE provided consistently high-quality portfolios, and the quantum strategies compared favorably with classical heuristics in terms of solution quality, especially under computational constraints. The study validates *Qiskit*'s practicality in real-world financial optimization workflows and highlights its readiness for hybrid quantum applications.

3.4.2 Risk Management

Effective risk management identifies, quantifies, and mitigates potential losses from market swings, credit events, and operational uncertainties. Classical techniques like Value at Risk (VaR) [94] and CVaR [95] estimate tail-risk, but struggle with (1) modeling high-dimensional asset correlations, causing exponential computational blowup, and (2) running large-scale Monte Carlo or nested optimizations when portfolios span hundreds or thousands of positions. *Qiskit*'s *Finance* module provides tools to approximate VaR and CVaR via amplitude estimation. This reduces sample complexity quadratically in comparison to classical Monte Carlo while capturing complex covariance structures through multi-qubit entanglement.

CVaR is defined as:

$$\text{CVaR} = \frac{1}{\alpha} \sum_{L_i \geq L_0} p_i L_i, \quad \alpha \in (0, 1), \quad (38)$$

where α is the tail-probability level (for instance, $\alpha = 0.05$ for the worst 5%), L_i denotes the i th discrete loss outcome, p_i is the probability of observing loss L_i (with $\sum_i p_i = 1$), and L_0 is VaR at level α (i.e., the smallest loss such that $\sum_{L_i \geq L_0} p_i = \alpha$). This represents the expected loss in the worst α -fraction of cases, providing coherent tail monitoring beyond VaR. In QC, each discrete loss scenario L_i is mapped to a basis state $|i\rangle$ with amplitude proportional to $\sqrt{p_i}$, preparing the “loss-probability” state:

$$|\psi_{\text{loss}}\rangle = \sum_{i=0}^{N-1} \sqrt{p_i} |i\rangle, \quad (39)$$

where N is the number of discrete scenarios and p_i are the normalized probabilities for each loss outcome. Amplitude estimation then projects onto all $|i\rangle$ where $L_i \geq L_0$ (the VaR threshold), yielding an estimate of CVaR. A variational ansatz, parameterized by portfolio-weight angles θ , is inserted after the loss-preparation circuit, and VQE minimizes the CVaR expectation $\langle \psi(\theta) | \hat{E}_{\text{CVaR}} | \psi(\theta) \rangle$ by adjusting θ , thus directly targeting tail-risk minimization. Given below, is the code snippet for CVaR-VQE algorithm:

```

1 # Discretize loss outcomes and probabilities
2 losses = [L1, L2, ..., LN] # Array of loss values
3 probs = [p1, p2, ..., pN] # Corresponding probabilities
4
5 # Build Loss Distribution state |psi_loss> = sum_i sqrt(p_i) |i>
6 loss_dist = LossDistribution(losses, probs)
7 loss_prep = loss_dist.construct_circuit()
8

```

```

9 # Define confidence level and create CVaR expectation operator
10 alpha = 0.05 # Worst-5% tail
11 cvar_exp = CVaRExpectation(loss_dist, alpha)
12
13 # Create variational ansatz parametrized by theta (portfolio weights)
14 n_qubits = loss_dist.num_qubits
15 ansatz = TwoLocal(n_qubits, 'ry', 'cz', reps=2)
16
17 # Configure VQE to minimize <psi(theta)|E_CVaR|psi(theta)>
18 vqe = VQE(ansatz,
19           optimizer=COBYLA(maxiter=100),
20           expectation=cvar_exp,
21           quantum_instance=AerSimulator())
22
23 # Run VQE on the combined circuit: loss preparation + ansatz
24 result = vqe.compute_minimum_eigenvalue(loss_prep)
25 optimal_angles = result.optimal_point

```

In lines 1–3, the arrays `losses` and `probs` are defined (with $\sum_i p_i = 1$). Then, `LossDistribution(losses, probs).construct_circuit()`, in lines 5–7 produces a `QuantumCircuit` that prepares $|\psi_{\text{loss}}\rangle = \sum_i \sqrt{p_i} |i\rangle$. Internally, *Qiskit* allocates $\lceil \log_2 N \rceil$ qubits and places all necessary rotations and entangling gates automatically, and no manual `QuantumRegister` calls or per-gate coding are required. Next, in lines 9–15, $\alpha = 0.05$ and `CVaRExpectation(loss_dist, alpha)` build the operator projecting onto $|i\rangle$ with $L_i \geq L_0$. Retrieving `n_qubits = loss_dist.num_qubits`, `TwoLocal(n_qubits, ry, cz, reps=2)` generates a two-layer ansatz. We then introduce parameterized Ry rotations and CZ gates without manual indexing. In lines 17–25, we compose the *loss-prep* and *ansatz* circuits internally, and run amplitude estimation to evaluate CVaR, and iterate classical updates to return `result.optimal_point` as the optimal angles θ .

As an output of this code, we get the circuit shown in Fig. 9. The first layer of $R_y(\phi_i)$ rotations followed by CZ-gates between qubit 0 and qubit 1 (and between qubit 2 and qubit 3) prepares the *loss-probability* state $|\psi_{\text{loss}}\rangle = \sum_i \sqrt{p_i} |i\rangle$. Here, each ϕ_i encodes $\sqrt{p_i}$, and the CZs entangle pairs of qubits such that measurement in the computational basis yields the correct distribution of losses. All qubit allocations and gate placements in this loss-prep subcircuit are handled automatically by *Qiskit*’s *LossDistribution*. The next two layers implement the variational *ansatz* from `twoLocal` with `reps = 2`. Four $R_y(\theta_{0..3})$ rotations on qubits 0–3 encode the portfolio-weight parameters θ_i , followed by CZs between qubits 0–1 and 2–3 to capture correlations in the ansatz state. Finally, a second block of four $R_y(\theta_{4..7})$ rotations refines the variational state. The final measurement projects onto the basis outcomes. This in turn provides the necessary information to VQE via amplitude estimation to evaluate and minimize CVaR, with all low-level details managed internally by *Qiskit*’s `TwoLocal` and `VQE` classes.

Below we review existing research that employs *Qiskit* for risk assessment in finance. The studies below illustrate how *Qiskit*’s built-in modules and hybrid quantum-classical workflows have been used to model loss distributions, estimate tail-risk, and optimize risk measures for complex portfolios. These examples provide a foundation for understanding the range of approaches and tools available in *Qiskit* for financial risk analysis.

Wilkens and Moorhouse [96] implemented end-to-end VaR and CCR workflows using *Qiskit*. They began by discretizing each risk-factor return via *Qiskit*’s `NormalDistribution`, loading a 3-qubit register for each factor and then applying piecewise linear transformations G_k to encode empirical marginals into qubit amplitudes:

$$\tilde{P}_{\text{norm}} = G_1(Z_1) + b G_2(Z_2), \quad (40)$$

where Z_1, Z_2 are standard normal registers and b scales the second factor to match portfolio sensitivities. The `NormalDistribution` circuit allocates the required q qubits (e.g., $q = 3$) and applies the necessary rotations and entangling gates to prepare the correlated normal state. Transformed registers are summed into a result register using built-in ripple-add circuits, and a comparator flags states with total loss below the VaR threshold. Iterative Quantum Amplitude Estimation (IQAE) then computes the VaR quantile with $O(\epsilon^{-1})$ scaling via *Qiskit*’s `AmplitudeEstimation` module.

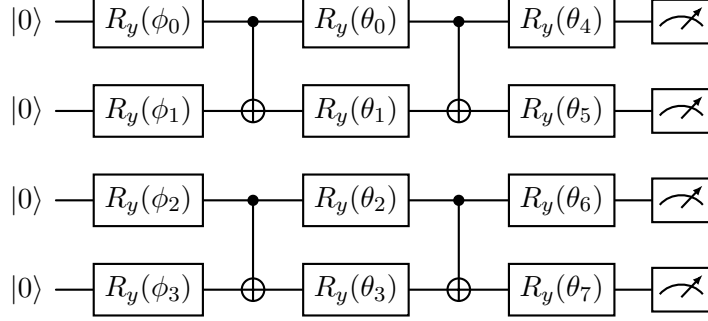


Figure 9: CVaR optimization circuit composed of a loss-probability state preparation layer followed by a two local ansatz with `reps` = 2. The first layer of $R_y(\phi_i)$ rotations and CZ -gates prepares the weighted superposition encoding $|\psi_{\text{loss}}\rangle = \sum_i \sqrt{p_i} |i\rangle$, representing portfolio losses. Subsequent $R_y(\theta_i)$ layers and entangling gates form the variational ansatz used in VQE to minimize CVaR.

For counterparty credit risk, two time-step registers (each built via `NormalDistribution` and piecewise transforms) are loaded and exposures accumulated using *Qiskit*’s arithmetic primitives. A final comparator calculates Potential Future Exposure (PFE), and IQAE yields the CCR measure. On a noise-free simulator with $q = 3$ per factor, the 99% VaR is -6.22 ($\approx 14\%$ off classical -5.44), and CCR PFE errors remain under 10% at $q = 3$, showing that *Qiskit*’s automatic qubit allocation, distribution loading, and built-in amplitude estimation enable these risk computations on current devices.

Dri *et al.* [97] implemented a multi-factor credit-risk model entirely within *Qiskit*. The authors first loaded each systemic risk factor Z_i into separate registers using *Qiskit*’s `NormalDistribution` and piecewise linear transforms, and then combined them via a weighted sum:

$$Y = \sum_{i=1}^R \alpha_i Z_i, \quad (41)$$

where each α_i was encoded into rotation angles by *Qiskit*’s controlled-rotation primitives. The resulting register Y was used to drive conditional rotations on asset qubits, encoding each asset’s default probability in its amplitude. *Qiskit*’s built-in arithmetic circuits (such as ripple-add and comparators) automatically summed losses and flagged those states below a VaR threshold, while the IQAE routine was accessed via *Qiskit*’s `AmplitudeEstimation` module, extracted the VaR quantile with $O(\epsilon^{-1})$ scaling. On a noise-free simulator using nine qubits, the 95% VaR matched classical baselines within a few percent, and CCR PFE errors remained under 10% (at $q = 3$ per factor), demonstrating that *Qiskit*’s automatic register allocation, distribution loading, and amplitude-estimation subroutines enabled practical quantum risk assessment despite limited hardware.

Grossi *et al.* [98] demonstrated an end-to-end QSVM-based fraud-detection pipeline using *Qiskit*. They constructed a quantum feature map $U_\phi(x)$ (e.g., a ZZ -feature map) by applying H -gate and CZ -rotations to embed each transaction’s normalized features into qubit amplitudes. They used *Qiskit*’s `ZZFeatureMap` class to handle all qubit allocations and gate placements automatically. The resulting quantum kernel was evaluated with

$$k(x, y) = |\langle 0^{\otimes n} | U_\phi(x)^\dagger U_\phi(y) | 0^{\otimes n} \rangle|^2, \quad (42)$$

where x and y are input feature vectors (each of dimension n), $U_\phi(x)$ is the parameterized feature-map unitary encoding transaction x , and n is the number of qubits used in the feature map. The `QuantumKernel` and `QSVM` classes handle the kernel matrix construction and solver routine, and no manual circuit-by-circuit coding is needed. After selecting optimal subsets of features via a quantum feature-selection loop, they trained and tested the QSVM on both state-vector and noise-modeled backends. The quantum pipeline achieved accuracy and Area Under the Curve (AUC) comparable to classical methods on a dramatically reduced dataset (for instance, 1500

training samples). This underscores that *Qiskit*’s automatic distribution of registers, feature-map circuits and kernel evaluation routines enabled practical, near-term quantum fraud-detection experiments.

Bharattej *et al.* [99] implemented QSVM and VQC pipelines for HFT risk classification entirely within *Qiskit*. They encoded each transaction’s normalized feature vector x_i into a quantum state via *Qiskit*’s `ZZFeatureMap`, which automatically allocated n qubits and inserted the necessary H -gate and CZ -rotation. The quantum kernel was defined as:

$$K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2, \quad (43)$$

where x_i and x_j are input feature vectors of dimension n , and $|\phi(x)\rangle = U_\phi(x)|0^{\otimes n}\rangle$ is the feature-map state prepared on those n qubits. *Qiskit*’s `QuantumKernel` and `QSVM` classes handled all kernel-matrix computations and the SVM optimization routine internally. For VQC, they employed *Qiskit*’s `VariationalQuantumClassifier` with a two-layer parameterized ansatz using *Qiskit* to automatically allocate qubits and place gates. On a noise-free simulator, QSVM achieved 91.3% accuracy, while VQC reached 92.7%, demonstrating that *Qiskit*’s built-in feature-map, kernel estimation, and variational-circuit modules enabled efficient risk-assessment workflows in high-frequency trading.

de Pedro *et al.* [100] implemented a quantum-enhanced VaR estimation workflow in *Qiskit* by restricting attention to a “window” of the loss distribution where the 1 % tail likely lies. They prepared a truncated Gaussian state using *Qiskit*’s state-preparation routines. Behind the scenes, `initialize()` (or related *Qiskit* state-preparation methods) generated the sequence of R_y and CX -gate needed to approximate $|\psi\rangle$ on those n qubits without any manual `QuantumRegister()` calls. They then computed the corresponding classical index-to-value mapping via:

$$\{0, \dots, 2^n - 1\} \mapsto \text{low} + \frac{\text{high} - \text{low}}{2^n - 1} i, \quad (44)$$

where i indexes the computational basis state $|i\rangle$, such that measurement $|\psi\rangle$ would yield a value within $[\text{low}, \text{high}]$. Here, $\text{low} = -3\sigma$ and $\text{high} = -2\sigma$ were chosen based on the portfolio’s overall standard deviation σ , and $i \in \{0, \dots, 2^n - 1\}$.

Once the truncated distribution was loaded, they ran the circuit on *Qiskit*’s `StatevectorSimulator` (and later IBM Lagos), obtaining a histogram whose most likely state ‘1101’ yielded a VaR of -9111.50 USD on four qubits. The true 1% VaR is defined as

$$\text{VaR} = \inf\{x : P[X \leq x] \geq \alpha\}, \quad \alpha = 0.01. \quad (45)$$

where α is the confidence level at which the VaR is computed. VaR at confidence level α is the smallest value of x such that the cumulative probability up to x is at least α . Here, X is the random variable representing the distribution. Since only the tail segment was targeted, they rescaled α to

$$\beta = \frac{\alpha - [1 - \text{CDF}(0, 1, 3)]}{\text{CDF}(0, 1, 2) - \text{CDF}(0, 1, 3)}, \quad (46)$$

where $\text{CDF}(0, 1, x)$ is the standard normal Cumulative Distribution Function (CDF) with mean $\mu = 0$ and standard deviation $\sigma = 1$. *Qiskit*’s state-preparation and `execute` calls handled all qubit allocation, amplitude loading, and measurement. On hardware, noise distorted the histogram, but a NN adjusted rotation parameters to reduce VaR error (USD 1,376 on five qubits) to under 1% relative to the reference (USD 1,380). This shows that *Qiskit*’s automatic circuit generation and backend integration, combined with NN noise mitigation, enabled accurate VaR estimation for portfolios of up to 139 assets.

Giri *et al.* [101] proposed a sequential affine-transformation approach for computing portfolio returns within *Qiskit*. They defined a normalized portfolio $P = [a_1, a_2, \dots, a_N]$ with $N = 2^m$ and set $\Psi = [a_1, a_2]$ as the initial one-qubit state, with subsequent blocks B_k containing the next 2^k assets of P . At each iteration k , they applied a H -gate supported add/sub operation—implemented via *Qiskit*’s built-in state-preparation and controlled-rotation

routines between the current amplitude register and B_k . This process doubled the qubit count at step k , forming a $(k + 1)$ -qubit system whose amplitudes represent the signed sums of selected asset subsets. Each H -gate supported operation scaled amplitudes by $1/2$, yielding a final amplitude vector

$$F(i_0, i_1, \dots, i_{m-1}) = (-1)^{i_0} a_{1+i_0} + \sum_{r=1}^{m-1} (-1)^{i_r} a_{2^r + \sum_{s=0}^{r-1} 2^{r-1-s} i_s} . \quad (47)$$

where $i = (i_0, i_1, \dots, i_{m-1}) \in \{0, 1\}^m$ indexes the binary qubit outcomes, m is the number of iterative affine-transformation steps (so that the register has $m + 1$ qubits), and a_j denotes the j -th asset value in the portfolio. The inner sum index r runs over each block size 2^r , and s indexes the previous contributions when forming the offset $2^r + \sum_{s=0}^{r-1} 2^{r-1-s} i_s$. *Qiskit*'s built-in `initialize` and conditional-initialization routines automatically allocated qubits and prepared the affine-transformation circuits. They measured the $(m + 1)$ -qubit register on a noise-free simulator (with 10^4 shots), and reproduced the exact combinatorial distribution of portfolio returns for $N = 8$ assets. On IBM Lagos hardware, optimized transpilation and measurement-error mitigation reduced noise effects such that the most probable bitstrings still corresponded to the largest returns, demonstrating scalable, hardware-executed risk assessment for portfolios of up to $N = 16$ assets.

4 Conclusion

In this paper, we present a structured and detailed survey of *Qiskit*, focusing on its role in enabling practical QC across diverse domains. We explore its modular architecture, including components such as *Terra*, *Aer*, and *Ignis*, and examine how its domain-specific libraries support real-world applications in cryptography and cybersecurity, image and signal processing, climate and energy, and finance. By analyzing published implementations and technical workflows, we highlight how *Qiskit* facilitates the transition from theoretical quantum models to executable circuits on both simulators and real quantum devices. The paper categorizes existing research into thematic areas, allowing readers to trace how *Qiskit* has been used to address domain-specific problems using hybrid quantum-classical approaches. This consolidation not only helps in understanding the breadth of *Qiskit*'s impact but also provides a reference point for researchers aiming to build upon established methods. Through this work, we aim to offer a comprehensive overview of *Qiskit*'s current capabilities and its significance in advancing QC research, development, and education.

5 List of Abbreviations

AUC	Area Under Curve
B5G	Beyond 5G
BB84	Bennett-Brassard 1984
BER	Bit Error Rate
BPSK	Binary Phase-Shift Keying
CCR	Counterparty Credit Risk
CDF	Cumulative Distribution Function
CDLSTM	Climate Deep Long-Short Term Memory
CHSH	Clauser–Horne–Shimony–Holt
CNN	Convolutional Neural Network
COBYLA	Constrained Optimization by Linear Approximation
CR_y	Controlled R_y -Gate

CT	Computed Tomography
CVaR	Conditional Value at Risk
CVaR-VQE	Conditional Value at Risk-Variational Quantum Eigensolver
CX-gate	Controlled X-Gate
CZ-gate	Controlled Z-Gate
DE	Differential Evolution
DFIG	Doubly-Fed Induction Generator
FRQI	Flexible Representation of Quantum Images
F_t	Forget Gate
H-gate	Hadamard Gate
HHL	Harrow-Hassidim-Lloyd
IBM	International Business Machines
IQAE	Iterative Quantum Amplitude Estimation
IQFT	Inverse Quantum Fourier Transform
KPS	Key-Pool Sharing
LSTM	Long Short-Term Memory
M-gate	Measurement Gate
MCMC	Markov Chain Monte Carlo
MD5	Message Digest Algorithm 5
MEV	Maximal Extractable Value
MILP	Mixed-Integer Linear Programming
MIMO	Maximum-Input Maximum-Output
ML	Machine Learning
MRI	Magnetic Resonance Imaging
MSE	Mean Squared Error
NFT	Nakanishi Fujii Todo
NIST	National Institute of Standards and Technology
NISQ	Noisy Intermediate-Scale Quantum
NN	Neural Networks
NOMA	Non-Orthogonal Multiple Access
OFDM	Orthogonal Frequency Division Multiplexing
p-Value	Probability Value
PCG	Permuted Congruential Generator
PFE	Potential Future Exposure
PO	Portfolio Optimization
PQK	Projected Quantum Kernel
PRNG	Pseudo-Random Number Generation
QAOA	Quantum Approximate Optimization Algorithm
QC	Quantum Computing
QCL	Quantum Circuit Learning
QCNN	Quantum Convolutional Neural Network
QFOQ	Quantum Finance Optimization Qubits

QFT	Quantum Fourier Transform
QKD	Quantum Key Distribution
QML	Quantum Machine Learning
QMO	Quantum Manifold Optimization
QNN	Quantum Neural Network
Q-OFDM	Quantum Orthogonal Frequency Division Multiplexing
QPE	Quantum Phase Estimation
QPP	Quantum Permutation Pad
QRNG	Quantum Random Number Generation
QSVC	Quantum Support Vector Classifier
QSVM	Quantum Support Vector Machines
QUBO	Quadratic Unconstrained Binary Optimization
RMSE	Root Mean-Squared Error
R_y	Rotation around y-axis
SHA	Secured Hash Algorithm
SLSQP	Sequential Least Squares Programming Optimizer
SPSA	Simultaneous Perturbation Stochastic Approximation
SVM	Support Vector Machine
TLKPS	Two-Level Key-Pool Sharing
UN-SDG	United Nations Sustainable Development Goals
USD	United States Dollar
VaR	Value at Risk
VQC	Variational Quantum Classifier
VQE	Variational Quantum Eigensolver
VQR	Variational Quantum Regressor
X-gate	Pauli-X Gate or Quantum NOT Gate

References

- [1] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [2] Peter W Shor. “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer”. In: *SIAM review* 41.2 (1999), pp. 303–332.
- [3] Peter W Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. In: *Proceedings 35th annual symposium on foundations of computer science*. Ieee. 1994, pp. 124–134.
- [4] Lov K Grover. “A fast quantum mechanical algorithm for database search”. In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 1996, pp. 212–219.
- [5] Ali Javadi-Abhari et al. “Quantum computing with Qiskit”. In: *arXiv preprint arXiv:2405.08810* (2024).
- [6] Mark Fingerhuth, Tomáš Babej, and Peter Wittek. “Open source software in quantum computing”. In: *PloS one* 13.12 (2018), e0208561.
- [7] Kaustubh Badrike et al. “Qisdax: An open source bridge from qiskit to trapped-ion quantum devices”. In: *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Vol. 1. IEEE. 2023, pp. 825–836.

- [8] M Emre Sahin et al. “Qiskit Machine Learning: an open-source library for quantum machine learning tasks at scale on quantum hardware and classical simulators”. In: *arXiv preprint arXiv:2505.17756* (2025).
- [9] Vlad Stirbu and Tommi Mikkonen. “Quantum Software Ecosystem: Stakeholders, Interactions and Challenges”. In: *International Conference on Software Business*. Springer. 2023, pp. 471–477.
- [10] Zeki C Seskir et al. “Quantum games and interactive tools for quantum technologies outreach and education”. In: *Optical Engineering* 61.8 (2022), pp. 081809–081809.
- [11] Thomas Alexander et al. “Qiskit pulse: programming quantum computers through the cloud with pulses”. In: *Quantum Science and Technology* 5.4 (2020), p. 044006.
- [12] Abha Satyavan Naik et al. “From portfolio optimization to quantum blockchain and security: A systematic review of quantum computing in finance”. In: *Financial Innovation* 11.1 (2025), pp. 1–67.
- [13] Kushal Batra et al. “Quantum machine learning algorithms for drug discovery applications”. In: *Journal of chemical information and modeling* 61.6 (2021), pp. 2641–2647.
- [14] Anshul Saxena et al. *Financial Modeling Using Quantum Computing*. Packt Publishing, 2023.
- [15] Kyung-Kyu Ko and Eun-Sung Jung. “Development of cybersecurity technology and algorithm based on quantum computing”. In: *Applied Sciences* 11.19 (2021), p. 9085.
- [16] Siddhartha Patra et al. “Efficient tensor network simulation of IBM’s largest quantum processors”. In: *Physical Review Research* 6.1 (2024), p. 013326.
- [17] Nathan Shammah et al. “Open hardware solutions in quantum technology”. In: *APL Quantum* 1.1 (2024).
- [18] Jules Tilly et al. “The variational quantum eigensolver: a review of methods and best practices”. In: *Physics Reports* 986 (2022), pp. 1–128.
- [19] Leo Zhou et al. “Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices”. In: *Physical Review X* 10.2 (2020), p. 021067.
- [20] Joshua J Tom et al. “Quantum computers and algorithms: a threat to classical cryptographic systems”. In: *Int. J. Eng. Adv. Technol* 12.5 (2023), pp. 25–38.
- [21] Zach Kirsch and Ming Chow. “Quantum computing: The risk to existing encryption methods”. In: *Retrieved from URL: [http://www. cs. tufts. edu/comp/116/archive/fall2015/zkirsch. pdf](http://www.cs.tufts.edu/comp/116/archive/fall2015/zkirsch.pdf)* (2015).
- [22] Renato Renner. “Security of quantum key distribution”. In: *International Journal of Quantum Information* 6.01 (2008), pp. 1–127.
- [23] William K Wootters and Wojciech H Zurek. “A single quantum cannot be cloned”. In: *Nature* 299.5886 (1982), pp. 802–803.
- [24] Adrian Kent. “Quantum state readout, collapses, probes, and signals”. In: *Physical Review D* 103.6 (2021), p. 064061.
- [25] Peter W Shor and John Preskill. “Simple proof of security of the BB84 quantum key distribution protocol”. In: *Physical review letters* 85.2 (2000), p. 441.
- [26] Tapaswini Mohanty et al. “An experimentally validated feasible quantum protocol for identity-based signature”. In: *Sādhanā* 50.1 (2025), pp. 1–11.
- [27] Ricardo Faleiro and Manuel Goulão. “Device-independent quantum authorization based on the Clauser-Horne-Shimony-Holt game”. In: *Physical Review A* 103.2 (2021), p. 022430.
- [28] Rafael Pereira da Silva. “Quantum Circuits with Qiskit”. In: *Available at SSRN 4359664* (2023).
- [29] Prodipto Das. “Design and Comparative Analysis of Quantum Hashing Algorithms using Qiskit”. In: *Authorea Preprints* (2023).

- [30] Robert Wille, Rod Van Meter, and Yehuda Naveh. “IBM’s Qiskit tool chain: Working with and developing for real quantum computers”. In: *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE. 2019, pp. 1234–1240.
- [31] R Kuang and M Perepechaenko. “Quantum encryption and decryption in IBMQ systems using quantum permutation pad”. In: *J. Commun* 17.12 (2022), pp. 972–978.
- [32] Miguel Herrero-Collantes and Juan Carlos Garcia-Escartin. “Quantum random number generators”. In: *Reviews of Modern Physics* 89.1 (2017), p. 015004.
- [33] Koichi Miyamoto and Kenji Shiohara. “Reduction of qubits in a quantum algorithm for Monte Carlo simulation by a pseudo-random-number generator”. In: *Physical Review A* 102.2 (2020), p. 022424.
- [34] Nday Kabulo Sinai and Hoh Peter In. “Q-RTOP: Quantum-Secure Random Transaction Ordering Protocol for Mitigating Maximal Extractable Value Attacks in Blockchains With a Priority Gas-Fee Policy”. In: *IEEE Access* 12 (2024), pp. 10036–10046.
- [35] Julie Germain, Ram Dantu, and Mark Thompson. “Qubit Reset and Refresh: A Gamechanger for Random Number Generation”. In: *Proceedings of the Twelfth ACM Conference on Data and Application Security and Privacy*. 2022, pp. 367–369.
- [36] Vaishnavi Kumar et al. “Quantum true random number generation on IBM’s cloud platform”. In: *Journal of King Saud University-Computer and Information Sciences* 34.8 (2022), pp. 6453–6465.
- [37] Yuanhao Li et al. “Quantum random number generator using a cloud superconducting quantum computer based on source-independent protocol”. In: *Scientific Reports* 11.1 (2021), p. 23873.
- [38] Francisco Orts et al. “A quantum circuit to generate random numbers within a specific interval”. In: *EPJ Quantum Technology* 10.1 (2023), p. 17.
- [39] Marcin M Jacak et al. “Quantum generators of random numbers”. In: *Scientific Reports* 11.1 (2021), p. 16108.
- [40] Shyam R Sihare. “Transformation of classical to quantum image, representation, processing and noise mitigation”. In: *International Journal of Image, Graphics and Signal Processing* 12.5 (2022), p. 10.
- [41] F. Yan, S. E. Venegas-Andraca, and K. Hirota. “Toward implementing efficient image processing algorithms on quantum computers”. In: *Soft Computing* 27.18 (2023), pp. 13115–13127.
- [42] Ahmed Elaraby. “Quantum medical images processing foundations and applications”. In: *IET Quantum Communication* 3.4 (2022), pp. 201–213.
- [43] Ashwini Shewale et al. “High Performance Computation Analysis for Medical Images using High Computational Methods”. In: *Proceedings of the Second International Conference on Information and Communication Technology for Competitive Strategies*. 2016, pp. 1–6.
- [44] Bobak Toussi Kiani, Agnes Villanyi, and Seth Lloyd. “Quantum medical imaging algorithms”. In: *arXiv preprint arXiv:2004.02036* (2020).
- [45] James CL Chow. “Quantum Computing in Medicine”. In: *Medical Sciences* 12.4 (2024), p. 67.
- [46] Naveen Jeyaraman et al. “Revolutionizing healthcare: the emerging role of quantum computing in enhancing medical technology and treatment”. In: *Cureus* 16.8 (2024).
- [47] Phuc Q Le, Fangyan Dong, and Kaoru Hirota. “A flexible representation of quantum images for polynomial preparation, image compression, and processing operations”. In: *Quantum Information Processing* 10 (2011), pp. 63–84.
- [48] Qiskit. *Qiskit Textbook*. 2024. URL: [https://github.com/Qiskit/textbook/tree/main/notebooks/ch-applications#\(accessed%20on%20April%202025\)](https://github.com/Qiskit/textbook/tree/main/notebooks/ch-applications#(accessed%20on%20April%202025)).
- [49] Pierre Decoodt et al. “Hybrid classical–quantum transfer learning for cardiomegaly detection in chest x-rays”. In: *Journal of imaging* 9.7 (2023), p. 128.

- [50] S Sofana Reka et al. “Exploring quantum machine learning for enhanced skin lesion classification: A comparative study of implementation methods”. In: *IEEE Access* (2024).
- [51] M. R. Almasaoodi et al. “A novel quantum OFDM transmission scheme”. In: *International Journal of Advanced Computer Science and Applications (IJACSA)* 15.5 (2024), pp. 440–449.
- [52] Abdulbasit MA Sabaawi, Mohammed R Almasaoodi, and Sándor Imre. “Exploiting OFDM method for quantum communication”. In: *Quantum Information Processing* 23.7 (2024), p. 256.
- [53] Shaik Anjmoon et al. “Hybrid Quantum-Classical Approaches to Optimize Signal Processing in Massive MIMO Arrays”. In: *2024 International Conference on Communication, Computer Sciences and Engineering (IC3SE)*. IEEE. 2024, pp. 1–6.
- [54] Juping Zhang et al. “Hybrid Quantum-Classical Neural Networks for Downlink Beamforming Optimization”. In: *IEEE Transactions on Wireless Communications* 23.11 (2024), pp. 16498–16512. DOI: 10.1109/TWC.2024.3442091.
- [55] H. Urgelles et al. “Quantum-based ML Detection in MIMO-NOMA for 6G”. In: *Quantum Reports* 6.4 (2024), pp. 533–549.
- [56] Getuar Rexhepi, Hyeon Seok Rou, and Giuseppe Thadeu Freitas de Abreu. *Quantum Manifold Optimization: A Design Framework for Future Communications Systems*. 2025. arXiv: 2504.09667 [eess.SP]. URL: <https://arxiv.org/abs/2504.09667>.
- [57] Salik Siddiqui et al. “Framework for Implementing Quantum Neural Networks in Wireless Communications”. In: *IEEE Access* 13 (2025), pp. 91655–91670. DOI: 10.1109/ACCESS.2025.3572416.
- [58] Param Pathak et al. “Resource Allocation Optimization in 5G Networks Using Variational Quantum Regressor”. In: *2024 International Conference on Quantum Communications, Networking, and Computing (QCNC)*. 2024, pp. 101–105. DOI: 10.1109/QCNC62729.2024.00025.
- [59] Ahmed Al-Shafei, Hamidreza Zareipour, and Yankai Cao. “A Review of High-Performance Computing and Parallel Techniques Applied to Power Systems Optimization”. In: *arXiv preprint* (July 2022). Submitted on 6 Jul 2022. arXiv: arXiv:2207.02388 [eess.SY]. URL: <https://arxiv.org/abs/2207.02388>.
- [60] Sam McArdle et al. “Quantum computational chemistry”. In: *Reviews of Modern Physics* 92.1 (2020), p. 015003.
- [61] Matthew Harrigan et al. *Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets*. <https://qiskit.org/documentation/nature/>. 2021.
- [62] M. Schwabe et al. “Opportunities and challenges of quantum computing for climate modelling”. In: *arXiv preprint arXiv:2502.10488* (2025).
- [63] K. Yu et al. “Comparison of Amplitude Estimation Algorithms by Implementation”. In: *arXiv preprint arXiv:2005.10258* (2020).
- [64] Arijit Choudhury et al. “CDLSTM: A Deep Learning Approach for Climate Forecasting in the Himalayan Region”. In: *Computers, Materials & Continua* 71.2 (2022), pp. 3183–3199. DOI: 10.32604/cmc.2022.022249.
- [65] Alessandro Sebastianelli et al. “On Circuit-Based Hybrid Quantum Neural Networks for Remote Sensing Imagery Classification”. In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 15 (2022), pp. 565–580. DOI: 10.1109/JSTARS.2021.3134785.
- [66] Rushir Bhavsar et al. “Classification of Potentially Hazardous Asteroids Using Supervised Quantum Machine Learning”. In: *IEEE Access* 11 (2023), pp. 75829–75848. DOI: 10.1109/ACCESS.2023.3297498.

- [67] Danyal Maheshwari, Julia Pelzer, and Miriam Schulte. “Predicting Heat Plume Temperature and Spatial Location Using Quantum Convolutional Neural Networks”. In: *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCNC)*. IEEE, 2025, pp. 623–627. DOI: 10.1109/QCNC64685.2025.00103.
- [68] Manish Kumar Gupta, Michał Romaszewski, and Piotr Gawron. “Potential of Quantum Machine Learning for Processing Multispectral Earth Observation Data”. In: *Bulletin of the Polish Academy of Sciences Technical Sciences* (2023). DOI: 10.36227/techrxiv.21898902.v1.
- [69] Thilanka Munasinghe et al. “Assessment of Quantum ML Applicability for Climate Actions: Comparison of the Variational Quantum Classifier and the Quantum Support Vector Classifier with Classical ML Models”. In: *Proceedings of the 2024 IEEE International Conference on Big Data (BigData)*. Washington, DC, USA: IEEE, Dec. 2024. DOI: 10.1109/BigData62323.2024.10825181.
- [70] M. A. Khan et al. “A mixed integer linear programming model for minimum backbone grid”. In: *Frontiers in Energy Research* 10 (2022). URL: <https://www.frontiersin.org/articles/10.3389/fenrg.2022.1004861/full>.
- [71] F. R. Albogamy et al. “Real-Time Energy Management and Load Scheduling with Renewable Energy Integration in Smart Grid”. In: *Sustainability* 14.3 (2022), p. 1792. DOI: 10.3390/su14031792. URL: <https://doi.org/10.3390/su14031792>.
- [72] E. Farhi, J. Goldstone, and S. Gutmann. *A Quantum Approximate Optimization Algorithm*. arXiv preprint arXiv:1411.4028. 2014. URL: <https://arxiv.org/abs/1411.4028>.
- [73] Zefan Tang et al. “Quantum-Secure Networked Microgrids”. In: *2020 IEEE Power & Energy Society General Meeting (PESGM)*. 2020, pp. 1–5. DOI: 10.1109/PESGM41954.2020.9281884.
- [74] Yifan Zhou and Peng Zhang. “Noise-resilient quantum machine learning for stability assessment of power systems”. In: *IEEE Transactions on Power Systems* 38.1 (2022), pp. 475–487.
- [75] Samantha Koretsky et al. “Adapting quantum approximation optimization algorithm (QAOA) for unit commitment”. In: *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE. 2021, pp. 181–187.
- [76] Kai Han, Tianyu Huang, and Longzhi Yin. “Quantum parallel multi-layer monte carlo optimization algorithm for controller parameters optimization of doubly-fed induction generator-based wind turbines”. In: *Applied Soft Computing* 112 (2021), p. 107813.
- [77] Anuj Ajagekar and Fei You. “Quantum computing based hybrid deep learning for fault diagnosis in electrical power systems”. In: *Applied Energy* 303 (2021), p. 117628.
- [78] C. Drepper. *E.ON allies with IBM quantum to advance energy transition goals*. <https://www.eon.com/en/about-us/media/press-release/2021/2021-09-02-eon-allies-with-ibm-quantum.html>. Last accessed 01 November 2021. 2021.
- [79] J. Dragan. *Tennessee startup using quantum technology to optimize micro-siting for wind turbines*. <https://thequantuminsider.com/2020/03/03/tennessee-startups-wind-power-quantum-panacea/>. Last accessed 09 December 2021. 2020.
- [80] HPC Wire. *IonQ and Hyundai motor partner to use quantum computing to advance batteries*. <https://www.hpcwire.com/off-the-wire/ionq-and-hyundai-motor-partner-to-use-quantum-computing-to-advance-batteries/>. Last accessed: 23 January 2023. 2023.
- [81] Raphael A Auer et al. *Quantum computing and the financial system: opportunities and risks*. BIS, Bank for International Settlements, 2024.
- [82] IBM Qiskit. *Qiskit Documentation*. 2024. URL: (<https://qiskit-community.github.io/qiskit-finance/>).

- [83] Harry M Markowitz and G Peter Todd. *Mean-variance analysis in portfolio choice and capital markets*. John Wiley & Sons, 2000.
- [84] Sudhansu Kumar Mishra, Ganapati Panda, and Babita Majhi. “Prediction based mean-variance model for constrained portfolio assets selection using multiobjective evolutionary algorithms”. In: *Swarm and evolutionary computation* 28 (2016), pp. 117–130.
- [85] IBM Qiskit. *Qiskit Documentation*. 2024. URL: (https://qiskit-community.github.io/qiskit-finance/tutorials/01_portfolio_optimization.html).
- [86] Fred Glover, Gary Kochenberger, and Yu Du. “Quantum Bridge Analytics I: a tutorial on formulating and using QUBO models”. In: *4or* 17.4 (2019), pp. 335–371.
- [87] Gines Carrascal et al. “Differential evolution VQE for crypto-currency arbitrage. Quantum optimization with many local minima”. In: *Digital Signal Processing* 148 (2024), p. 104464.
- [88] Giuseppe Buonaiuto et al. “Best practices for portfolio optimization by quantum computing, experimented on real quantum devices”. In: *Scientific Reports* 13.1 (2023), p. 19434.
- [89] Mouhamad Abushaqra. “Quantum Finance Optimization Qubits (QFOQ): An Approach to Accelerate Financial Model Testing”. In: *Available at SSRN 5007468* (2024).
- [90] Saswati Soumya Tripathy, Neerja Koul, and Hemil Patel. “Comparative Study between Quantum and Classical Methods: Few Observations from Portfolio Optimization Problem”. In: *2022 International Conference on Trends in Quantum Computing and Emerging Business Technologies (TQCEBT)*. IEEE. 2022, pp. 1–5.
- [91] Romina Yalovetzky et al. “NISQ-HHL: Portfolio optimization for near-term quantum hardware”. In: *arXiv preprint arXiv:2110.15958* (2021).
- [92] Ginés Carrascal et al. “Backtesting quantum computing algorithms for portfolio optimization”. In: *IEEE Transactions on Quantum Engineering* 5 (2023), pp. 1–20.
- [93] Anton Robert et al. “Resource-efficient quantum algorithm for protein folding”. In: *npj Quantum Information* 7.1 (2021), p. 38.
- [94] Philip Best. *Implementing value at risk*. John Wiley & Sons, 2000.
- [95] Yinlam Chow et al. “Risk-sensitive and robust decision-making: a cvar optimization approach”. In: *Advances in neural information processing systems* 28 (2015).
- [96] Sascha Wilkens and Joe Moorhouse. “Quantum computing for financial risk measurement”. In: *Quantum Information Processing* 22.1 (2023), p. 51.
- [97] Emanuele Dri et al. “Towards practical quantum credit risk analysis”. In: *Journal of Physics: Conference Series*. Vol. 2416. 1. IOP Publishing. 2022, p. 012002.
- [98] Michele Grossi et al. “Mixed quantum–classical method for fraud detection with quantum feature selection”. In: *IEEE Transactions on Quantum Engineering* 3 (2022), pp. 1–12.
- [99] Rana Veer Samara Sihman Bharattej Rupavath et al. “Quantum Machine Learning for High-Frequency Trading and Risk Management”. In: *arXiv preprint* (2025).
- [100] Luis de Pedro et al. “VaR Estimation with Quantum Computing Noise Correction Using Neural Networks”. In: *Mathematics* 11.20 (2023), p. 4355.
- [101] Anish Giri, David Hyde, and Kálmán Varga. “A New Algorithm for Applying Sequences of Affine Transformations in Quantum Circuits”. In: *arXiv preprint arXiv:2412.09714* (2025). v2, submitted April 17, 2025. URL: <https://arxiv.org/abs/2412.09714>.